

“Flood Monitoring and Prediction using Satellite Data, Remote Sensing and Machine Learning Techniques”

A Project Report submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING

Submitted by

SATHWIK (VU22CSEN0102225)

SRAVYA (VU22CSEN0101228)

ABHISHEK (VU22CSEN0100158)

KISHAN (VU22CSEN0100385)

Under the esteemed guidance of

**Dr. Naina Narang
Associate Professor**



GITAM
DEEMED TO BE UNIVERSITY

**Department of Computer Science and Engineering
GITAM School of Computer Science and Engineering
GITAM (Deemed to be University)
Visakhapatnam
2026**

Department of Computer Science and Engineering
GITAM School of Computer Science and Engineering
GITAM (Deemed to be University)
Visakhapatnam



DECLARATION

We, hereby declare that the project report entitled Flood Monitoring and Prediction using Satellite Data, Remote Sensing and Machine Learning Techniques is an original work done in the Department of Computer Science and Engineering, GITAM (Deemed to be University) submitted in partial fulfillment of the requirements for the award of the degree of B.Tech. in Computer Science and Engineering. The work has not been submitted to any other college or University for the award of any degree or diploma.

Date:

Registration No(s)	Name(s)	Signature
VU22CSEN0102225	SATHWIK	
VU22CSEN0101228	SRAVYA	
VU22CSEN0100158	ABHISHEK	
VU22CSEN0100385	KISHAN	

Department of Computer Science and Engineering
GITAM School of Computer Science and Engineering
GITAM (Deemed to be University)
Visakhapatnam



CERTIFICATE

This is to certify that the project report entitled Flood Monitoring and Prediction using Satellite Data, Remote Sensing and Machine Learning Techniques is a bonafide record of work carried out by SATHWIK - VU22CSEN0102225, SRAVYA - VU22CSEN0101228, ABHISHEK - VU22CSEN0100158, KISHAN - VU22CSEN0100385 students submitted in partial fulfillment of requirement for the award of degree of Bachelors of Technology in Computer Science and Engineering)

Date :

Project Guide

Dr. Naina Narang

Head of the Department

ACKNOWLEDGEMENT

Dr. Naina Narang guided this project from the beginning, and her input genuinely shaped how it turned out. She gave us clear feedback when we were on the wrong track and pushed us to think more carefully about the work. We're grateful for that.

Prof. Pragnyaban Mishra and Dr. N. Suresh Kumar reviewed the project at different stages. Their questions were pointed and their suggestions practical both helped us close gaps we hadn't noticed ourselves.

We'd also like to thank our faculty and department for the resources and support that made this possible.

TABLE OF CONTENTS

1. ABSTRACT	1
2. INTRODUCTION	2
2.1 BACKGROUND.....	2
2.2 THE CASE FOR SAR-BASED MONITORING	2
2.3 DEEP LEARNING FOR SAR FLOOD MAPPING	3
3. LITERATURE REVIEW	4
4. PROBLEM IDENTIFICATION & OBJECTIVES	9
4.1 PROBLEM STATEMENT	9
4.2 CHALLENGES.....	9
4.3 OBJECTIVES	10
5. EXISTING SYSTEM	11
5.1 CONVENTIONAL FLOOD MONITORING APPROACHES	11
5.2 OPTICAL SATELLITE IMAGERY	11
5.3 THRESHOLD-BASED SAR FLOOD DETECTION	11
5.4 DRAWBACKS OF EXISTING APPROACHES.....	12
6. PROPOSED SYSTEM	13
6.1 SYSTEM OVERVIEW	13
6.2 ADVANTAGES OVER EXISTING METHODS.....	14
6.3 FUNCTIONAL REQUIREMENTS	14
7. SYSTEM ARCHITECTURE	15
7.1 ARCHITECTURAL OVERVIEW	15
7.2 BLOCK 1 — DATA ACQUISITION	15
7.3 BLOCK 2 — PREPROCESSING AND NORMALIZATION	16
7.4 BLOCK 3 — U-NET MODEL	16
7.5 BLOCK 4 — POST-PROCESSING.....	18
7.6 BLOCK 5 — AREA QUANTIFICATION AND CHANGE DETECTION.....	18
7.7 BLOCK 6 — RISK ASSESSMENT.....	18
7.8 BLOCK 7 — OUTPUT GENERATION.....	19
8. TOOLS/TECHNOLOGIES USED.....	20
8.1 PROGRAMMING LANGUAGE — PYTHON 3.10	20
8.2 DEEP LEARNING — TENSORFLOW 2.X AND KERAS.....	20

8.3 GEOSPATIAL PROCESSING — RASTERIO	20
8.4 SATELLITE DATA PLATFORM — GOOGLE EARTH ENGINE.....	21
8.5 IMAGE PROCESSING — SCIKIT-IMAGE	21
8.6 AUGMENTATION — ALBUMENTATIONS	21
8.7 NUMERICAL COMPUTING — NUMPY AND PANDAS	21
8.8 VISUALIZATION — MATPLOTLIB AND FOLIUM.....	22
8.9 DEVELOPMENT PLATFORM — GOOGLE COLAB AND DRIVE	22
8.10 METEOROLOGICAL DATA — OPEN-METEO API.....	22
9. IMPLEMENTATION & RESULTS	24
9.1 PHASE I — MODEL TRAINING ON SEN1FLOODS11	24
9.1.1 DATASET PREPARATION	24
9.1.2 PREPROCESSING IMPLEMENTATION.....	25
9.1.3 DATA PIPELINE.....	27
9.1.4 TRAINING RESULT.....	30
9.2 PHASE I — TEST SET EVALUATION	32
9.3 PHASE II — VIJAYAWADA DATA ACQUISITION.....	35
9.4 PHASE II — BINARY FLOOD MASK GENERATION	37
9.5 PHASE II — FLOOD EXTENT ANALYSIS.....	39
9.6 FLOOD AREA COMPARISON AND ANALYSIS	42
9.7 RAINFALL ANALYSIS	44
9.8 RISK ASSESSMENT RESULTS	46
10.CONCLUSION.....	49
10.1 SUMMARY OF CONTRIBUTIONS	49
10.2 LIMITATIONS	49
10.3 FUTURE WORK	50
11.REFERENCES	51

1. ABSTRACT

Floods are among the most devastating natural calamities, causing significant damage to life, structures, and economy, especially in flood-prone locations. Traditional flood monitoring techniques such as ground surveys and optical satellite images often face problems during extreme weather events due to low visibility and delayed data. In this project, a flood monitoring and prediction system using Deep Learning and SAR satellite image analysis is presented. A U-Net model is built and trained to detect flood zones accurately based on satellite SAR images. The system will analyze satellite data in various time frames and produce flood maps indicating the area covered by floods. On top of flood zone detection, the system uses rainfall data to perform risk assessment by correlating the amount of precipitation and the extent of flood areas. This will allow us to categorize areas depending on their risk level. The system presented here presents a way to effectively automate flood monitoring and prediction processes through a combination of remote sensing, environmental analysis, and deep learning technologies.

2. INTRODUCTION

2.1 BACKGROUND

Every year, floods kill more people than any other weather event roughly 40% of all weather-related deaths worldwide and cost hundreds of billions in damages. India gets hit especially hard. The major peninsular rivers swell fast during the southwest monsoon, and when heavy upstream rainfall, dam releases, and drainage failures line up at the same time, a river system can blow past capacity in hours. The September 2024 Vijayawada flood was a clear example: the Srisailem catchment soaked up more rain than usual, Prakasam Barrage released at record volumes, and the Budameru drain swelled simultaneously. Within 48 hours, a city of nearly a million people was underwater.

The monitoring failure was just as striking. River gauging stations record discharge but can't show where water is actually spreading across the city. Normally, optical satellites like Sentinel-2 would fill that gap but monsoon cloud cover made nearly every acquisition over Vijayawada in September 2024 useless. Emergency responders were working off scattered phone reports and drone footage from isolated areas. Nobody had a city-wide picture.

2.2 THE CASE FOR SAR-BASED MONITORING

Synthetic Aperture Radar works when optical sensors can't. Instead of measuring reflected sunlight, SAR sends out microwave pulses and records what bounces back and microwaves go straight through clouds. Rain doesn't matter. Overcast skies don't matter. ESA's Sentinel-1 satellites run at C-band (~5.4 GHz), with 10-meter resolution and a 6-day revisit per satellite. Two satellites combined means South Asia gets covered roughly every 3 days. It's also free.

The flood detection logic is relatively straightforward: open water scatters microwaves away from the satellite rather than back toward it, so it shows up dark in SAR imagery. Rough terrain, buildings, and vegetation bounce energy back and appear bright. That contrast holds across both VV and VH polarization channels. Using both channels together helps it's easier to avoid false detections from shadows or smooth rooftops when you have more signal to work with.

2.3 DEEP LEARNING FOR SAR FLOOD MAPPING

In open fields, you can map floods with basic thresholding: call everything below a backscatter cutoff "water" and move on. Cities are harder. Shadows, asphalt, and dense building clusters produce low backscatter too, and they can look a lot like water if you're only looking at one pixel at a time.

Neural networks specifically convolutional encoder-decoder models can use spatial context to sort that out. A dark pixel surrounded by bright urban returns is probably a shadow or a road, not a flooded street. Trained on enough examples, these models draw cleaner flood boundaries than thresholding can.

The U-Net architecture has become standard for satellite image segmentation. Its skip connections bring spatial detail from early layers forward into the final prediction, which matters when you're trying to pin down exactly where the water ends and the dry street begins. Sen1Floods11, published by Google researchers, provides globally distributed, labeled SAR flood data built specifically for training and benchmarking this kind of model.

3. LITERATURE REVIEW

S. No.	Paper Details	Summary	Techniques / Algorithms	Research Gap
1	Ronneberger, Fischer & Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI, 2015	Introduces the U-Net encoder-decoder architecture with skip connections for precise image segmentation. Became the foundation for most subsequent flood mapping models.	U-Net, CNN, encoder-decoder, skip connections	Designed for biomedical images, not SAR. Sensitive to satellite image noise and needs large labeled datasets.
2	Zortea et al., "Flood Mapping Using Sentinel-1 Images and Lightweight U-Nets Trained on Synthesized Events," IEEE GRSL, 2023	Trains a lightweight U-Net on synthetically generated Sentinel-1 flood events, reducing dependence on real labeled data while keeping the model fast enough for near-real-time use.	Lightweight U-Net, synthetic data generation, SAR preprocessing, augmentation	Synthetic data doesn't fully capture real flood variability. Geographic generalization is limited, and the lightweight design trades some accuracy for speed.
3	Mehta & Rawat, "Monitoring Flash Flood Using Optical and Microwave Sensor Dataset," DICCT, 2023	Fuses optical and SAR data to improve flash flood detection under cloudy conditions, validated on a real	Optical + SAR fusion, SNAP processing, multi-sensor analysis	Optical data still fails under heavy cloud cover. The fusion workflow isn't

	.	event in Assam, India.		automated and doesn't scale beyond the study region.
4	Nguyen et al., "Estimating Uncertainty in Flood Model Outputs Using Machine Learning Informed by Monte Carlo Analysis," MIGARS, 2024	Combines ML with Monte Carlo simulations to quantify uncertainty in flood model outputs, improving the reliability of predictions for decision-making.	Machine learning, Monte Carlo simulation, uncertainty modeling	Computationally expensive due to repeated simulations. Too slow for real-time deployment.
5	Liu et al., "Integration of Remote Sensing and Crowdsourced Data for Fine-Grained Urban Flood Detection," IEEE JSTARS, 2024	Combines satellite imagery with crowdsourced reports to improve spatial resolution and real-time flood detection in dense urban areas.	Data fusion, crowdsourced integration, machine learning, remote sensing	Crowdsourced data is noisy and inconsistent. Privacy concerns around user-generated inputs, and performance depends heavily on data availability.
6	Chendeb El Rai et al., "Multi-Source Attention-Based Fusion for Segmentation of Natural Disasters," IEEE GRSL, 2024	Proposes an attention-based fusion framework that weights features from multiple data	Attention mechanism, multi-source fusion, deep learning	High computational overhead makes real-time deployment difficult.

	.	sources to improve segmentation accuracy in complex flood scenarios.	segmentation	Requires large multi-source datasets.
7	Sudiana et al., "Performance Evaluation of 3D CNN for Multitemporal Flood Classification With SAR Data," IEEE JSTARS, 2025	Uses a 3D CNN to process multitemporal SAR image stacks, capturing temporal change patterns to improve flood classification over time.	3D CNN, multitemporal SAR, temporal feature extraction	High memory and compute requirements. Large temporal training datasets needed, limiting practical deployment.
8	Huang et al., "WaterDetectionNet: A New Deep Learning Method for Flood Mapping With SAR CNN," IEEE JSTARS, 2024	Introduces WaterDetectionNet, a CNN architecture for SAR-based flood mapping that handles complex terrain and improves segmentation accuracy over standard approaches.	CNN, SAR segmentation, feature extraction	Needs substantial labeled training data. Performance varies across terrain types, and the model offers limited interpretability.
9	Asadi et al., "Enhanced-HisSegNet: Improved SAR Flood Segmentation With Learnable Histogram Layers," IEEE GRSL, 2025	Integrates learnable histogram layers with active contour models to sharpen flood boundary detection and	Learnable histogram layers, active contour model, deep	Complex architecture with high computational cost. Not suitable for

	.	handle difficult water-land transitions in SAR imagery.	learning, SAR feature enhancement	real-time use and requires careful hyperparameter tuning.
10	Chung et al., "Exploring Flooded Fraction Prediction Through ML Focusing on Medical Infrastructure," BigComp, 2025	Uses ML models to predict flood-affected fractions with a focus on identifying vulnerable medical infrastructure in Southeast US coastal areas.	Machine learning, flood prediction, risk assessment, infrastructure analysis	Limited to a specific geographic region. Model generalization to other environments is not demonstrated.
11	Nava et al., "Artisanal and Small-Scale Mine Detection in Semi-Desertic Areas by Improved U-Net," IEEE GRSL, 2022	Applies an improved U-Net to detect small-scale mining areas in remote sensing imagery, demonstrating segmentation models' broader utility in environmental monitoring.	Improved U-Net, CNN, image segmentation	Not a flood detection study. Adaptability to flood mapping scenarios is untested.
12	Iqbal, Anghel & Datcu, "Coastline Extraction From SAR Data Using Doppler Centroid Images," IEEE GRSL, 2022	Extracts coastline boundaries from SAR Doppler centroid images, improving water-land boundary	SAR processing, Doppler centroid analysis, image	Limited to coastal areas. Not applicable to inland flood detection.

	.	detection in coastal zones.	segmentation	
13	Li et al., "UNet Combined With Attention Mechanism for Extracting Flood Submerged Range," IEEE JSTARS, 2022	Adds a CBAM attention mechanism to U-Net to improve flood segmentation, particularly for edge detection and small inundated areas.	U-Net, CBAM attention, deep learning, SAR processing	Attention adds computational cost and model complexity. Still requires large labeled datasets.
14	Roth et al., "Estimating Ensemble Likelihoods for the Sentinel-1 Based Global Flood Monitoring Product," IEEE JSTARS, 2023	Describes the Copernicus EMS global flood monitoring system, which combines multiple detection algorithms into an ensemble and produces uncertainty estimates alongside flood maps.	Ensemble methods, SAR processing, change detection, Bayesian inference	Combining multiple models makes the system hard to interpret and too slow for real-time decision-making.

4. PROBLEM IDENTIFICATION & OBJECTIVES

4.1 PROBLEM STATEMENT

Effective disaster response depends on knowing where the water is. Two tools dominate flood monitoring optical satellites and river gauging networks and both tend to break down exactly when they're needed most. Optical imagery from Sentinel-2, Landsat, and MODIS gets wiped out by cloud cover during active monsoon events. Gauging stations measure discharge at fixed locations but can't show how far water has spread across a city or farming area.

The September 2024 Vijayawada flood put a face on this problem. APSDMA officials had no real-time picture of which neighborhoods were underwater, how deep it was, or whether levels were rising or falling. That missing information slowed rescue deployment and made it harder to issue targeted evacuation orders. So this isn't purely a technical problem it's a gap between what satellite technology can deliver and what authorities on the ground actually receive during a crisis.

The question driving this project: using free Sentinel-1 SAR data and publicly labeled flood imagery, can a system be built that maps flood extent accurately enough to be operationally useful and can it assess risk by folding in rainfall context?

4.2 CHALLENGES

Several things make this harder than it sounds.

- Cloud cover is the obvious one. The monsoon conditions that cause flooding are the same ones that make optical satellites useless. SAR handles cloud cover fine, but brings its own headaches speckle noise, missing values at scene edges, and backscatter behavior in dense urban areas that throws off simple threshold-based detection.
- Class imbalance is trickier. Floodwater might cover less than 5-15% of a given SAR scene. A naively trained model learns that predicting "no flood" everywhere gives a decent accuracy score, and does exactly that. It performs well on paper and fails in the field.
- Permanent water bodies are a quieter problem. The Krishna River is always there, flood or no flood. Without masking it out, the river channel gets counted as floodwater in every analysis period and inflates the numbers.
- And flood extent alone doesn't give decision-makers much to act on. Turning a flood mask into something useful means bringing in rainfall data that the SAR imagery doesn't contain.

4.3 OBJECTIVES

Three things this project is trying to do:

- **Deep learning optimization.** Train a U-Net on the Sen1Floods11 dataset to detect floods from Sentinel-1 SAR imagery. This means building a preprocessing pipeline that handles normalization, resizing, and noise and choosing loss functions that account for class imbalance, since standard cross-entropy will just predict dry land everywhere.
- **Multi-source integration.** SAR flood maps alone aren't enough. The system combines them with rainfall data to understand how floods behave over time, and uses auxiliary datasets to mask permanent water bodies so the extent numbers reflect actual inundation rather than baseline river coverage.
- **End-to-end monitoring system.** The full pipeline processes SAR imagery across pre-flood, active-flood, and post-flood phases, generates flood masks, calculates inundated area, and produces a risk rating based on rainfall intensity output that a disaster management team could actually use to make decisions.

5. EXISTING SYSTEM

5.1 CONVENTIONAL FLOOD MONITORING APPROACHES

India's state disaster management authorities mostly rely on three things: river gauges, weather station data, and post-flood field surveys. Gauges measure water level and discharge at fixed points useful for upstream warnings, but limited to exactly that. A reading at Prakasam Barrage tells an operator that dangerous volumes of water are being released. It says nothing about which neighborhoods flood first, or how fast.

Field surveys by APSDMA and NDRF teams can document inundation in detail, but only once the water starts pulling back. During an active flood, the places that most need documenting are the ones you can't safely access

5.2 OPTICAL SATELLITE IMAGERY

Optical satellites like Sentinel-2, Landsat 8/9, and Planet SkySat are widely used for water mapping. Water absorbs strongly in near-infrared and shortwave-infrared wavelengths, making it appear dark against vegetation and soil. The standard tool for picking out water from these images is NDWI ($\text{Green} - \text{NIR}$) / ($\text{Green} + \text{NIR}$) and it works well in many parts of the world.

Not during an Indian monsoon. Peak southwest monsoon conditions the same weeks that produce catastrophic floods push cloud cover over Andhra Pradesh above 80% for days at a stretch. Sentinel-2 revisits every 10 days. Landsat, every 16. With persistent overcast, there may be no usable acquisition for the entire duration of a flood event. Better algorithms don't fix this. When clouds block the surface, there's no signal

5.3 THRESHOLD-BASED SAR FLOOD DETECTION

Some agencies use Sentinel-1 SAR to get around the cloud problem. The basic approach compares pre-flood and flood-peak backscatter pixels where the ratio drops sharply get flagged as newly inundated. UNOSAT's Flood Detection Tool and the Copernicus Emergency Management Service both use versions of this.

It works up to a point. Thresholds calibrated for agricultural floodplains in Bangladesh don't transfer to dense urban areas like Vijayawada. Urban SAR scenes are full of things that look like water building shadows, smooth asphalt, flat concrete rooftops and pixel intensity alone can't tell them apart from actual floodwater. The method also requires a clean pre-flood

reference image from within a few weeks prior, under dry conditions. Given Sentinel-1's 6-day revisit and unpredictable weather, that's often not available when you need it.

5.4 DRAWBACKS OF EXISTING APPROACHES

The gaps across all these approaches follow a consistent pattern. Ground gauges can't map where water spreads. Field surveys are retrospective. Optical imagery disappears under monsoon clouds at exactly the wrong moment. Threshold SAR methods are site-specific, struggle in cities, and can't distinguish water from shadows by intensity alone. None of them separate newly flooded areas from permanent water bodies without someone doing it manually. And none produce anything a disaster manager can directly act on they generate maps, and then someone has to figure out what the maps mean.

That's what this project is trying to fix.

6. PROPOSED SYSTEM

6.1 SYSTEM OVERVIEW

The system is an automated flood monitoring and risk assessment pipeline in two phases.

Phase I trains a U-Net on Sen1Floods11 a globally distributed, labeled SAR flood dataset to build detection capability across diverse conditions. Phase II deploys the trained model on Vijayawada-specific Sentinel-1 composites and extends the output with permanent water removal, area quantification, temporal change detection, and rainfall-integrated risk scoring.

Training globally before applying locally is intentional. A model trained only on Krishna River floodplain imagery would learn the backscatter characteristics of that specific place and probably fail anywhere else. Sen1Floods11 exposes the model to varied flood shapes, land cover types, and SAR acquisition conditions first. The Vijayawada application comes after the model arrives with general knowledge and gets applied to a specific problem.

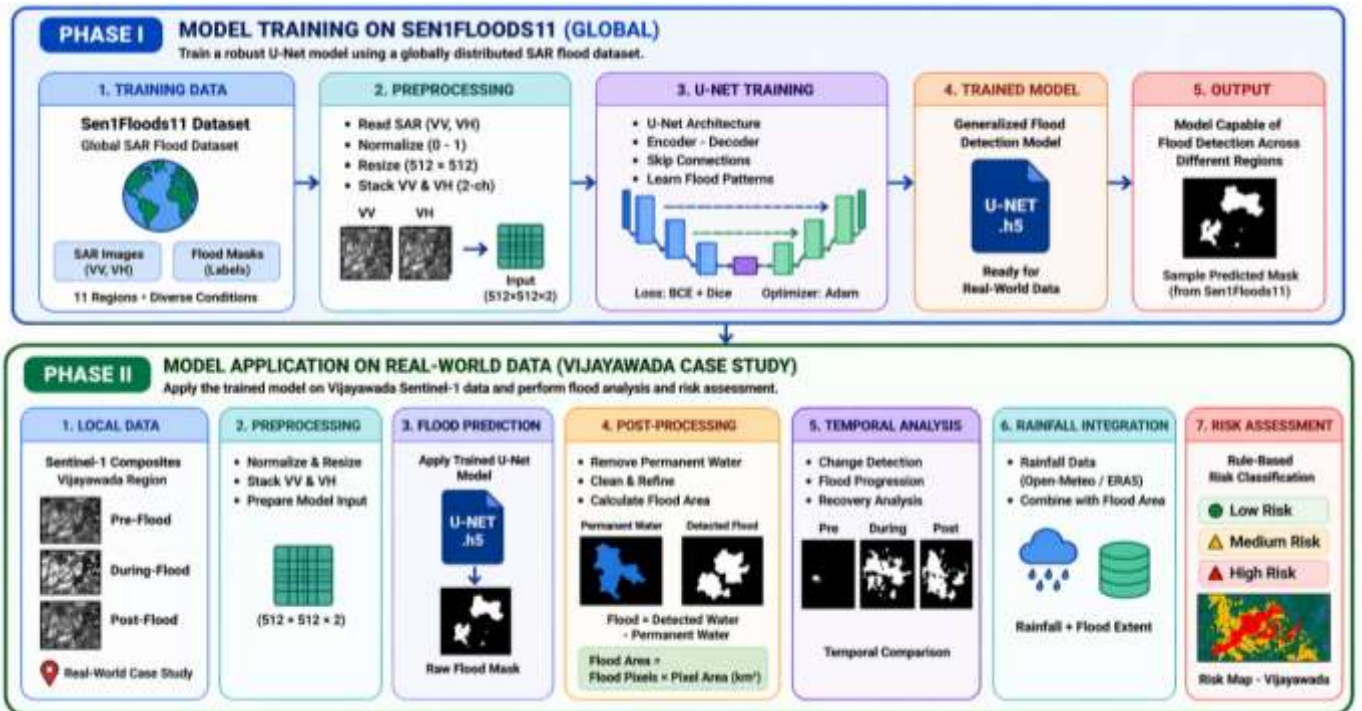


Figure 6.1: Two-phase framework for flood monitoring — Phase I (global training on Sen1Floods11) and Phase II (application on Vijayawada case study with risk assessment)

6.2 ADVANTAGES OVER EXISTING METHODS

- **Cloud cover is irrelevant.** Sentinel-1 operates at C-band and passes through monsoon clouds without issue. Flood maps get generated regardless of weather.
- **Urban scenes are handled.** The U-Net uses the spatial neighborhood of each pixel, not just its backscatter value in isolation. That's how it separates open water from building shadows and smooth rooftops something threshold methods can't do.
- **The Krishna River doesn't count as floodwater.** The JRC Global Surface Water mask is built into post-processing, automatically removing permanent water bodies before area calculations run.
- **The full event is tracked.** Pre-flood, during-flood, and post-flood composites let the system measure when flooding started, how far it spread at peak, and how quickly it pulled back.
- **The output is usable.** Flood extent combined with Open-Meteo rainfall data produces a risk classification per locality per time period something a disaster management team can act on directly, not just look at.
- **Everything is free.** Sentinel-1, Sen1Floods11, Open-Meteo all open access. Processing runs on open-source Python and Google Colab's free tier.

6.3 FUNCTIONAL REQUIREMENTS

The system handles seven core functions. Data acquisition pulls Sentinel-1 GRD IW composites in VV and VH polarization from Google Earth Engine for any bounding box and time window. Preprocessing cleans NaN and infinity values, normalizes VV to $[-30, 0]$ dB and VH to $[-35, -5]$ dB, and resizes patches to 512×512 pixels.

Training uses combined Dice + Binary Cross-Entropy loss with a class weight of 8.43 for flood pixels — necessary because water typically covers a small fraction of any scene — along with adaptive learning rate scheduling. The trained model generates per-pixel flood probability maps for locality-specific SAR crops, thresholded at 0.5 for binary masks.

Before any area calculations run, the JRC occurrence-based mask strips out permanent water. Risk classification then pulls historical daily rainfall from Open-Meteo and combines it with flood area statistics to produce a rating per locality per time period.

Final outputs: binary flood masks, probability heatmaps, change detection maps, flood area bar charts, and an interactive Folium risk map.

7. SYSTEM ARCHITECTURE

7.1 ARCHITECTURAL OVERVIEW

The system is a sequential pipeline with seven functional blocks: Data Acquisition, Preprocessing and Normalization, U-Net Model, Post-Processing, Area Quantification, Risk Assessment, and Output Generation. Each block has defined inputs and outputs, which makes the pipeline straightforward to test and modify independently.

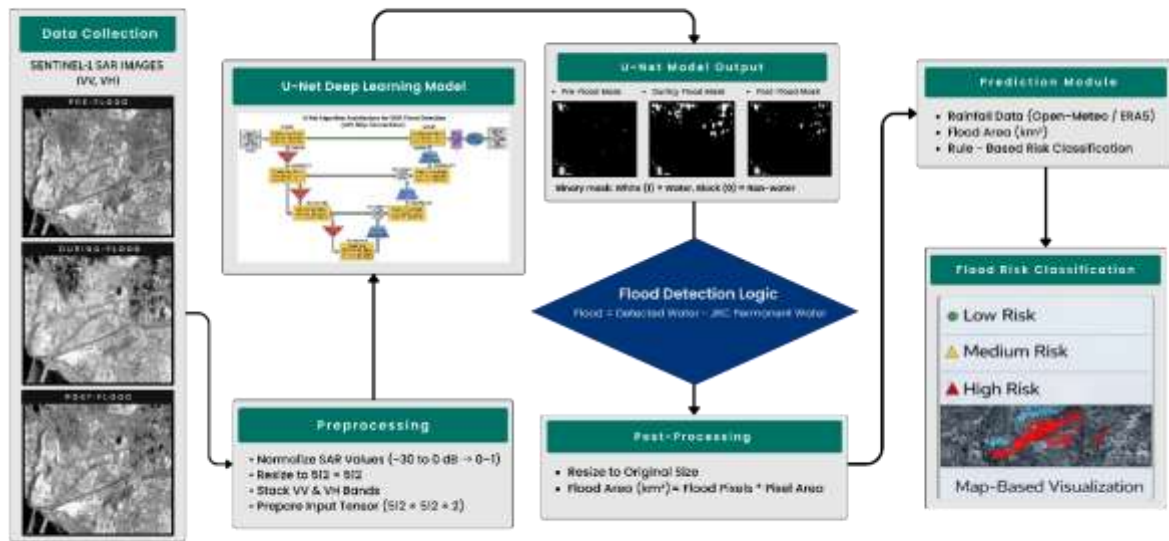


Figure 7.1: Seven-block system architecture showing data flow from SAR acquisition through risk assessment and output generation

7.2 BLOCK 1 DATA ACQUISITION

- Two external sources feed the system.
- The primary source is Google Earth Engine, which provides access to the Copernicus Sentinel-1 GRD image collection. The system filters by bounding box, date range, instrument mode (IW), and polarization (VV + VH), then computes a temporal mean composite to reduce speckle. Images are exported as two-band GeoTIFFs via `geemap.ee_export_image` at 30-meter resolution a compromise between the native 10-meter GRD resolution and GEE's 50 MB single-export limit.
- The secondary source is the Open-Meteo archive API, which returns ERA5 reanalysis daily precipitation data for Vijayawada (16.5193°N, 80.6305°E) without an API key. Daily precipitation_sum values are fetched via HTTP GET for the three analysis

periods and passed to the risk classification module.

- The JRC Global Surface Water dataset (JRC/GSW1_4/GlobalSurfaceWater) is also pulled from GEE. The occurrence band, thresholded at 80%, produces the permanent water mask used in post-processing

7.3 BLOCK 2 PREPROCESSING AND NORMALIZATION

- Preprocessing applies four operations to raw SAR data, identically during training on Sen1Floods11 and inference on Vijayawada crops.
- *NaN and infinity handling.* A validity mask is built as the element-wise AND of `np.isfinite(vv)` and `np.isfinite(vh)`. Invalid pixels are replaced with zero, and the mask is carried forward to zero out normalized output at nodata locations.
- *Band normalization.* VV backscatter is mapped from $[-30, 0]$ dB to $[0, 1]$ using `VV_norm = clip((VV + 30) / 30, 0, 1)`. VH is mapped from $[-35, -5]$ dB to $[0, 1]$ using `VH_norm = clip((VH + 35) / 30, 0, 1)`. These ranges cover the typical physical backscatter values for Sentinel-1 GRD data over land and water.
- *Spatial resampling.* Normalized arrays are resized to 512×512 pixels using bilinear interpolation for model input. Output masks are resized back to original dimensions using nearest-neighbor interpolation (`order=0`) to keep binary values intact.
- *Channel stacking.* VV and VH are stacked into a $(512, 512, 2)$ input tensor, expanded to $(1, 512, 512, 2)$ for batch inference.

7.4 BLOCK 3 U-NET MODEL

7.4.1 MODEL ARCHITECTURE

The model takes $(512, 512, 2)$ inputs and outputs a $(512, 512, 1)$ flood probability map. Three types of modular blocks make up the architecture:

Convolutional block. Two consecutive Conv2D layers with 3×3 kernels, same padding, and He Normal initialization, each followed by Batch Normalization and ReLU. Optional Dropout is added when a non-zero `dropout_rate` is set. Batch Normalization speeds up convergence and reduces sensitivity to initialization choices.

Encoder block. A convolutional block followed by 2×2 MaxPooling. Returns both the conv output (as a skip connection) and the pooled output (as input to the next stage).

Decoder.block. $2\times$ bilinear UpSampling2D, concatenation with the matching encoder skip connection, then a convolutional block. Bilinear upsampling produces cleaner boundaries than transposed convolutions and avoids checkerboard artifacts.

The encoder uses filter sizes of 64, 128, 256 (10% dropout), and 512 (20% dropout). The bottleneck runs 1024 filters with 30% dropout. The decoder mirrors the encoder in reverse: 512, 256, 128, 64. The output layer is Conv2D(1, 1, activation='sigmoid'), producing per-pixel flood probability. Total trainable parameters: ~ 31 million.

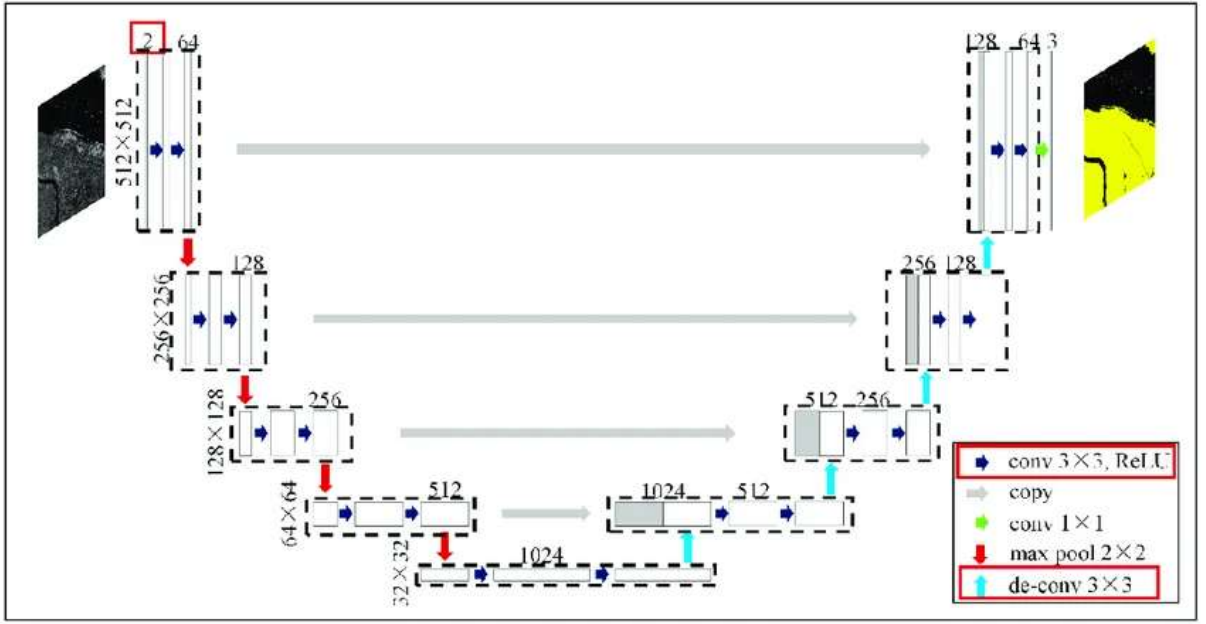


Figure 7.2: Detailed U-Net architecture showing encoder path (left), bottleneck, decoder path (right), and skip connections

7.4.2 LOSS FUNCTION AND TRAINING

The combined loss is: $\text{Loss} = 0.5 \times \text{BCE} + 0.5 \times \text{Dice_Loss}$

Binary Cross-Entropy handles per-pixel classification and is sensitive to local errors. Dice Loss optimizes the overlap between prediction and ground truth, which handles class imbalance better than BCE alone. Flood pixels receive a sample weight of 8.43 the ratio of non-flood to flood pixels across the training set applied via `model.fit(sample_weight)`

Training uses Adam ($\text{lr}=1\text{e-}4$), up to 50 epochs, with three callbacks: ModelCheckpoint saving the best model by `val_dice_coefficient`, EarlyStopping with `patience=8` and `best_weights` restoration, and ReduceLROnPlateau with `factor=0.5`, `patience=4`, and

$\text{min_lr}=1\text{e-}7$. Geometric augmentations — horizontal flip, vertical flip, 90° rotation - are applied to training samples only via Albumentations.

7.5 BLOCK 4 POST-PROCESSING

Two operations follow inference.

First, the binary prediction mask is aligned with the JRC permanent water mask and any pixel where $\text{jrc_mask} == 1$ is set to 0. This removes the Krishna River channel, Budameru drain, and other standing water features that are present year-round and shouldn't be counted as flood.

Second, flood pixels are converted to area. At 30-meter resolution, each pixel covers $900 \text{ m}^2 = 0.0009 \text{ km}^2$.

7.6 BLOCK 5 AREA QUANTIFICATION AND CHANGE DETECTION

Flood area is calculated for each locality-period combination and organized into summary tables. Change detection uses pixel-wise subtraction between adjacent period masks: Pre-to-During identifies where new flooding appeared, and During-to-Post shows where drainage occurred. These difference images are visualized with a diverging blue-white-red colormap red for new inundation, blue for recession.

7.7 BLOCK 6 RISK ASSESSMENT

Risk classification combines two inputs: a rainfall indicator computed as $0.7 \times \text{max_daily_rain} + 0.3 \times \text{mean_daily_rain}$, and the detected flood area per locality per period. A two-dimensional rule-based classifier assigns risk levels as follows:

Table 7.1: Risk classification rule table combining rainfall indicator and detected flood area

Rain Level	Flood Level	Risk Classification	Threshold Criteria
High (>65 mm/day)	High (>0.8 km ²)	High Risk	Both indicators at High
High	Moderate	High Risk	High rain, moderate flood
Moderate (35-65 mm/day)	High	High Risk	Moderate rain, large flood
Moderate	Moderate (0.4-0.8 km ²)	Medium Risk	Both at moderate
Low (<35 mm/day)	Low (<0.4 km ²)	Low Risk	Both indicators at Low

7.8 BLOCK 7 OUTPUT GENERATION

Outputs fall into four categories.

Spatial visualizations include binary flood masks (white = water), color-coded maps distinguishing new flood (red), permanent water (blue), and land (gray), probability heatmaps using a plasma colormap, and Pre-to-During and During-to-Post change maps.

Statistical graphics include grouped bar charts of flood area by locality and period, line charts showing temporal flood progression, and a 3×3 risk heatmap matrix.

An interactive Folium HTML map overlays results on a CartoDB Positron basemap with risk-colored markers for each locality. All figures are exported at 300 DPI.

8.TOOLS/TECHNOLOGIES USED

8.1 PROGRAMMING LANGUAGE - PYTHON 3.10

Python is the only language used across the entire project. The reason is practical: geospatial loading, array math, model training, and interactive map generation all live in the same ecosystem. Nothing needs to cross a language boundary. Everything runs in Google Colab's hosted Python 3.10 environment.

8.2 DEEP LEARNING - TENSORFLOW 2.X AND KERAS

The U-Net is defined, trained, and evaluated through TensorFlow's Keras API. Keras supplies the core layers directly Conv2D, BatchNormalization, Dropout, MaxPooling2D, UpSampling2D, Concatenate, Input and custom components slot in cleanly alongside them. The combined loss function (`weighted_dice_bce_loss`) and evaluation metrics (`dice_coefficient`, `iou_score`) are written as plain Python functions and registered through `custom_objects` so Keras can find them when reloading a saved model.

Training data flows through `tf.data`, which chains parallel loading, shuffling, batching, and prefetching. Prefetching in particular matters: it keeps the CPU preparing the next batch while the GPU is still processing the current one, so the GPU doesn't sit idle between steps. Training runs on Colab's NVIDIA T4 GPU (~16 GB VRAM), which handles a 31-million-parameter network at 512×512 without memory issues at reasonable batch sizes. The final model saves in `.keras` format a single file containing architecture, weights, and optimizer state.

8.3 GEOSPATIAL PROCESSING - RASTERIO

Rasterio reads and writes GeoTIFFs while keeping spatial metadata coordinate reference system, resolution, bounding box, nodata values intact alongside the pixel data. The result is a NumPy array that also knows where it is on Earth, which matters when cropping rasters to locality boundaries or overlaying outputs on a map.

Three specific uses: loading Sentinel-1 SAR composites and the JRC permanent water mask; converting geographic coordinates to pixel row/column indices via `src.index()` for locality-level cropping; and writing output flood masks back to GeoTIFF with the original spatial metadata preserved so downstream tools can place them correctly.

8.4 SATELLITE DATA PLATFORM - GOOGLE EARTH ENGINE

GEE provides server-side access to the Sentinel-1 GRD archive and JRC Global Surface Water dataset. Collection filtering, temporal compositing, and export configuration all run in GEE's infrastructure nothing computationally heavy happens locally. The geemap wrapper simplifies the last step: `geemap.ee_export_image()` handles authentication, format conversion, and file download, so the only thing that lands on the local machine is the finished composite GeoTIFF.

8.5 IMAGE PROCESSING - SCIKIT-IMAGE

`transform.resize` from `scikit-image` handles spatial resampling at two points. Inputs to the model resize to 512×512 via bilinear interpolation (`preserve_range=True`, `anti_aliasing=True`). Outputs from the model resize back to original locality dimensions via nearest-neighbor interpolation (`order=0`), which keeps flood/non-flood edges hard rather than blurred.

The `preserve_range=True` flag is easy to overlook but important. Without it, `scikit-image` silently rescales pixel values to $[0, 1]$ during resize which would corrupt SAR data that's already been normalized to a specific physical range in Block 2. The flag keeps values exactly as they are and makes Block 2 the single place where value scaling happens.

8.6 AUGMENTATION - ALBUMENTATIONS

Training augmentations: `HorizontalFlip` ($p=0.5$), `VerticalFlip` ($p=0.5$), `RandomRotate90` ($p=0.5$). `Albumentations`' key property is that it applies the same transformation to the SAR image and its flood label mask at the same time. This pairing is non-negotiable a flipped image with an unflipped label would actively mislead the model about where water is.

No intensity augmentations are used. SAR backscatter values encode real physical properties of the surface. Randomly shifting or stretching them would generate training samples that no actual Sentinel-1 acquisition would produce, introducing a distribution mismatch between training data and real inference inputs.

8.7 NUMERICAL COMPUTING - NUMPY AND PANDAS

NumPy is the substrate for everything: NaN detection (`np.isfinite`), clipping (`np.clip`), masking (`np.where`), flood pixel counting (`np.sum`), channel stacking (`np.stack`). Virtually every operation in the pipeline touches a NumPy array at some point.

Pandas manages the tabular side: `Sen1Floods11` train/validation/test CSV splits, rainfall DataFrames from Open-Meteo, flood area tables per locality and period, and risk classification outputs. Both are pre-installed in Colab.

8.8 VISUALIZATION - MATPLOTLIB AND FOLIUM

Matplotlib handles all static figures. Binary flood masks: gray colormap, vmin=0, vmax=1. Probability heatmaps: plasma colormap. Change detection: bwr diverging colormap — blue for drainage, white for no change, red for new inundation. Grouped bar charts break down flood area by locality and period. Line charts track how extent changed across the three periods. The 3×3 risk heatmap visualizes the rule table from Block 6. Everything saves at 300 DPI with `bbox_inches='tight'`.

The interactive HTML map comes from Folium, which wraps Leaflet.js. Flood masks convert to RGBA PNG overlays via PIL and load onto the map through `folium.raster_layers.ImageOverlay`, positioned at their correct geographic bounds. Circular markers on each locality open popups showing the risk level per period.

8.9 DEVELOPMENT PLATFORM - GOOGLE COLAB AND DRIVE

Matplotlib handles all static figures. Binary flood masks: gray colormap, vmin=0, vmax=1. Probability heatmaps: plasma colormap. Change detection: bwr diverging colormap — blue for drainage, white for no change, red for new inundation. Grouped bar charts break down flood area by locality and period. Line charts track how extent changed across the three periods. The 3×3 risk heatmap visualizes the rule table from Block 6. Everything saves at 300 DPI with `bbox_inches='tight'`.

The interactive HTML map comes from Folium, which wraps Leaflet.js. Flood masks convert to RGBA PNG overlays via PIL and load onto the map through `folium.raster_layers.ImageOverlay`, positioned at their correct geographic bounds. Circular markers on each locality open popups showing the risk level per period.

8.10 METEOROLOGICAL DATA - OPEN-METEO API

Open-Meteo serves ERA5 reanalysis precipitation data through a REST API with no key or registration required. For each of the three analysis periods, `daily_precipitation_sum` for Vijayawada's coordinates comes back as a JSON response, which parses straight into a Pandas DataFrame and flows into the risk classification module..

. Table 8.1: Summary of tools and technologies used in the project

Tool / Library	Version	Primary Use
Python	3.10	Core language
TensorFlow / Keras	2.14+	U-Net model and training pipeline
Rasterio	1.3+	GeoTIFF reading and coordinate transforms
Google Earth Engine API	0.1.380+	SAR composite download
Geemap	0.32+	GEE image export wrapper
scikit-image	0.21+	Image resizing for model I/O
Albumentations	1.3+	Paired SAR + mask augmentation
NumPy	1.24+	Array operations throughout
Pandas	2.0+	CSV splits and result tables
Matplotlib	3.7+	All static figures
Folium	0.14+	Interactive HTML flood map
Open-Meteo API	—	ERA5 rainfall data
Google Colab	—	GPU training environment
Google Drive	—	Persistent file storage

9. IMPLEMENTATION & RESULTS

9.1 PHASE I - MODEL TRAINING ON SEN1FLOODS11

9.1.1 DATASET PREPARATION

Sen1Floods11 downloads from Google Cloud Storage via gsutil 446 paired Sentinel-1 SAR images and hand-labeled flood masks covering diverse flood events across multiple continents. Each SAR image carries two polarization bands, VV and VH, which together give the model enough signal to separate water from non-water surfaces across varied terrain types.

The dataset ships with predefined CSV splits: roughly 252 images for training, 89 for validation, and 90 for testing. Using the official splits rather than random ones matters it keeps the evaluation clean and comparable to published baselines. Before any training starts, file paths are constructed from these CSVs and each SAR image is verified against its corresponding label file, catching any missing or mismatched pairs early.

```
=====
S1 SAR images   : 446
Flood masks     : 446
=====
Paired files     : 446
Missing masks    : 0

Sample S1 files:
Bolivia_103757_S1Hand.tif
Bolivia_129334_S1Hand.tif
Bolivia_195474_S1Hand.tif

Sample mask files:
Bolivia_103757_LabelHand.tif
Bolivia_129334_LabelHand.tif
Bolivia_195474_LabelHand.tif

Official splits:
Train : 252 images
Val   : 89 images
Test  : 90 images
```

Code Snippet 9.1.1: Dataset Download using gsutil:

```
# Download Sen1Floods11 Dataset (SAR images + flood masks)

import os

DATA_ROOT = "/content/sen1floods11"
os.makedirs(DATA_ROOT, exist_ok=True)

# Download SAR images and labels
!gsutil -m cp -r gs://sen1floods11/v1.1/data/flood_events/HandLabeled/* {DATA_ROOT}

# Download train/val/test split files
!gsutil -m cp -r gs://sen1floods11/v1.1/splits/flood_handlabeled/* {DATA_ROOT}

print("Dataset download complete!")
```

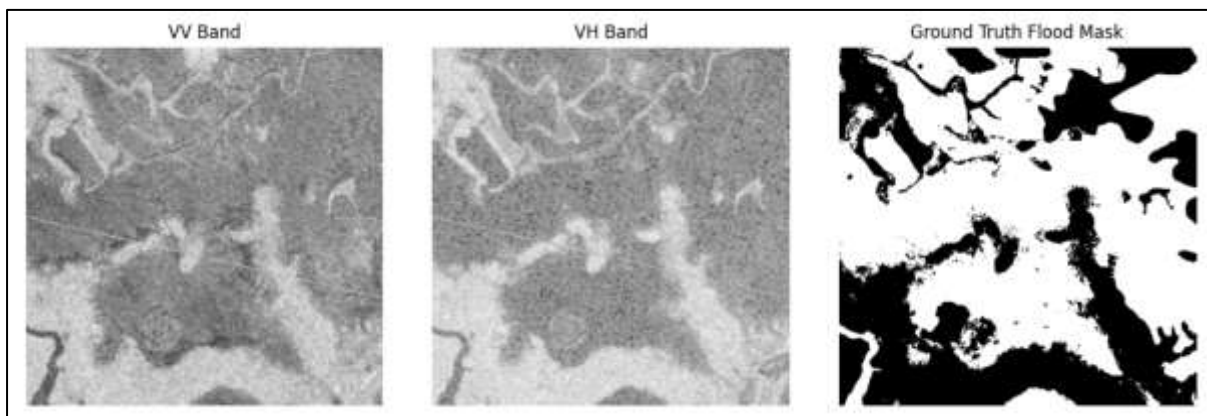


Figure 9.1: Sample Sen1Floods11 Training Images showing VV band, VH band, and corresponding ground truth flood mask.

The geographic spread of flood events in Sen1Floods11 across Asia, Africa, Europe, and the Americas is what makes it useful here. A model trained on it sees riverine flooding, urban inundation, and vegetated floodplains under different SAR acquisition conditions, which is exactly the kind of variety that produces a model capable of generalizing rather than memorizing one region's backscatter signature.

9.1.2 PREPROCESSING IMPLEMENTATION

Raw Sentinel-1 SAR images arrive with invalid pixels scattered throughout NaN and infinite values introduced by sensor limitations and acquisition geometry. Left unhandled, these corrupt gradients during training and destabilize inference, so the first thing the preprocessing pipeline does is deal with them.

Each image loads via `load_s1_image`, which reads VV and VH as float32 arrays and immediately builds a validity mask using `np.isfinite(vv) & np.isfinite(vh)`. Anything that fails that check gets zeroed out. The mask carries forward through the rest of preprocessing so those locations stay zeroed even after normalization invalid pixels don't get a rescaled value, they stay at zero throughout.

Normalization follows. VV maps from $[-30, 0]$ dB to $[0, 1]$ using `clip((VV + 30) / 30, 0, 1)`; VH maps from $[-35, -5]$ dB to $[0, 1]$ using `clip((VH + 35) / 30, 0, 1)`. These dB ranges cover the realistic physical backscatter values for Sentinel-1 GRD data over land and water clipping handles the occasional outlier without distorting the rest of the distribution. Applying normalization only to valid pixels and re-zeroing invalid locations afterward means the model never sees a rescaled noise value where there should be no data at all.

Code Snippet 9.1.2: SAR Image Loading and Normalization Functions:

```
import rasterio
import numpy as np

def load_s1_image(s1_path):
    """
    Load Sentinel-1 SAR image (VV & VH bands)
    and handle invalid values.
    """
    with rasterio.open(s1_path) as src:
        vv = src.read(1).astype(np.float32)
        vh = src.read(2).astype(np.float32)

    # Create valid mask
    valid = np.isfinite(vv) & np.isfinite(vh)

    # Replace NaN / inf with 0
    vv = np.where(valid, vv, 0.0)
    vh = np.where(valid, vh, 0.0)

    return vv, vh, valid

def normalize_bands(vv, vh, valid=None):
    """
    Normalize VV and VH bands to [0,1]
    """
    VV_MIN, VV_MAX = -30.0, 0.0
    VH_MIN, VH_MAX = -35.0, -5.0

    vv_norm = np.clip((vv - VV_MIN) / (VV_MAX - VV_MIN), 0.0, 1.0)
    vh_norm = np.clip((vh - VH_MIN) / (VH_MAX - VH_MIN), 0.0, 1.0)

    # Apply valid mask
    if valid is not None:
        vv_norm = np.where(valid, vv_norm, 0.0)
        vh_norm = np.where(valid, vh_norm, 0.0)

    return vv_norm, vh_norm
```

After normalization, VV and VH stack into a two-channel array shape (512, 512, 2) which becomes the model's input. Two channels matter because VV and VH respond differently to surface properties: using both gives the model more to work with when distinguishing open water from other low-backscatter surfaces like shadows or smooth rooftops.

A visualization check runs on a handful of training samples before any training starts. Each sample displays four panels side by side: normalized VV, normalized VH, the ground truth flood mask, and a VV-flood overlay that shows where the label sits relative to the backscatter pattern. This isn't just a sanity check on pixel values it confirms that the label masks are spatially aligned with the SAR imagery, that normalization hasn't clipped legitimate signal, and that the nodata regions are correctly zeroed rather than carrying forward garbage values.

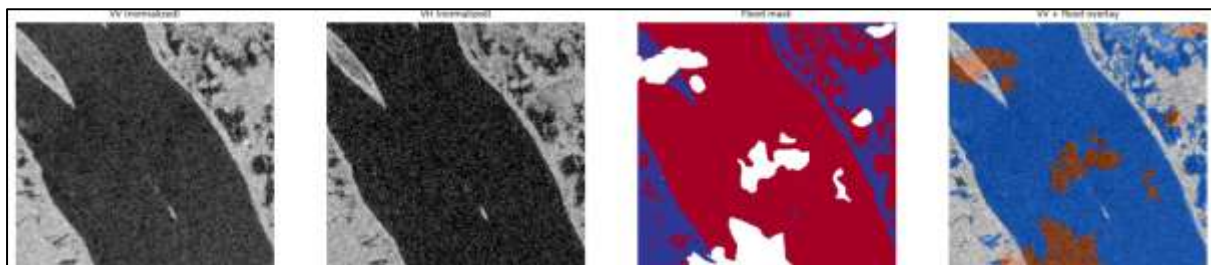


Figure 9.2: Preprocessed Sentinel-1 SAR data showing normalized VV band, VH band, ground truth flood mask, and overlay visualization of flood regions

The overlays in particular make it easy to spot whether the preprocessing is behaving correctly: flooded pixels should sit in visibly darker VV regions, and the mask boundary should line up cleanly with the backscatter transition rather than drifting into bright terrain.

9.1.3 DATA PIPELINE

The training pipeline runs through `tf.data`, which handles the full data flow loading, preprocessing, augmentation, batching, and prefetching in a single optimized chain.

File paths from the CSV splits feed into `tf.data.Dataset.from_tensor_slices()`, giving TensorFlow a dataset it can iterate over efficiently. Training data gets shuffled with a buffer of 300 before anything else happens, which randomizes the order the model sees examples and stops it from picking up on sequence patterns in the data.

NumPy-based operations SAR loading, normalization, augmentation can't run natively inside the TensorFlow graph, so they plug in through `tf.py_function`. The `load_and_preprocess` wrapper handles all three steps and returns the image, label, and sample weight tensor for each

example. Augmentation only runs on training data; validation and test sets go through loading and normalization only, keeping their evaluation consistent across epochs.

Batching at size 8 keeps GPU memory manageable for a 31-million-parameter network at 512×512. Prefetching with AUTOTUNE lets the CPU prepare the next batch while the GPU is still processing the current one without it, the GPU stalls between steps waiting for data. With it, data preparation and model computation run in parallel and GPU utilization stays high throughout training.

Sample weights travel through the pipeline alongside each image and label. Flood pixels carry a weight of 8.43; non-flood pixels carry 1.0. This means the optimizer sees a much larger effective penalty for missing a flood pixel than for a false positive which is the right trade-off for a disaster detection system.

Code Snippet 9.1.3: TensorFlow Data Pipeline Implementation

```
import tensorflow as tf
import numpy as np

BATCH_SIZE = 8
IMG_SIZE = 512
AUTOTUNE = tf.data.AUTOTUNE

def load_and_preprocess(s1_path, label_path, augment=False):
    """
    Loads SAR image + label, applies preprocessing and optional augmentation
    """
    img, lbl, wt = preprocess_image(
        s1_path.numpy().decode(),
        label_path.numpy().decode()
    )

    if augment:
        img, lbl, wt = augment_sample(img, lbl, wt)

    return img.astype(np.float32), lbl.astype(np.float32), wt.astype(np.float32)
```

```

def make_dataset(s1_paths, label_paths, augment=False,
                batch_size=BATCH_SIZE, shuffle=False):
    """
    Builds tf.data dataset pipeline
    """
    ds = tf.data.Dataset.from_tensor_slices((s1_paths, label_paths))

    if shuffle:
        ds = ds.shuffle(buffer_size=len(s1_paths), seed=42)

    def _load_fn(s1, lbl):
        img, lab, wt = tf.py_function(
            lambda a, b: load_and_preprocess(a, b, augment),
            [s1, lbl],
            [tf.float32, tf.float32, tf.float32]
        )

        img.set_shape([IMG_SIZE, IMG_SIZE, 2])
        lab.set_shape([IMG_SIZE, IMG_SIZE, 1])
        wt.set_shape([IMG_SIZE, IMG_SIZE, 1])

        return img, lab, wt

    ds = ds.map(_load_fn, num_parallel_calls=AUTOTUNE)
    ds = ds.batch(batch_size)
    ds = ds.prefetch(AUTOTUNE)

    return ds

# Create datasets
train_ds = make_dataset(train_s1, train_lbl, augment=True, shuffle=True)

val_ds = make_dataset(s1_paths=val_s1, label_paths=val_lbl, augment=False, shuffle=False)

test_ds = make_dataset(test_s1, test_lbl, augment=False, shuffle=False)

```

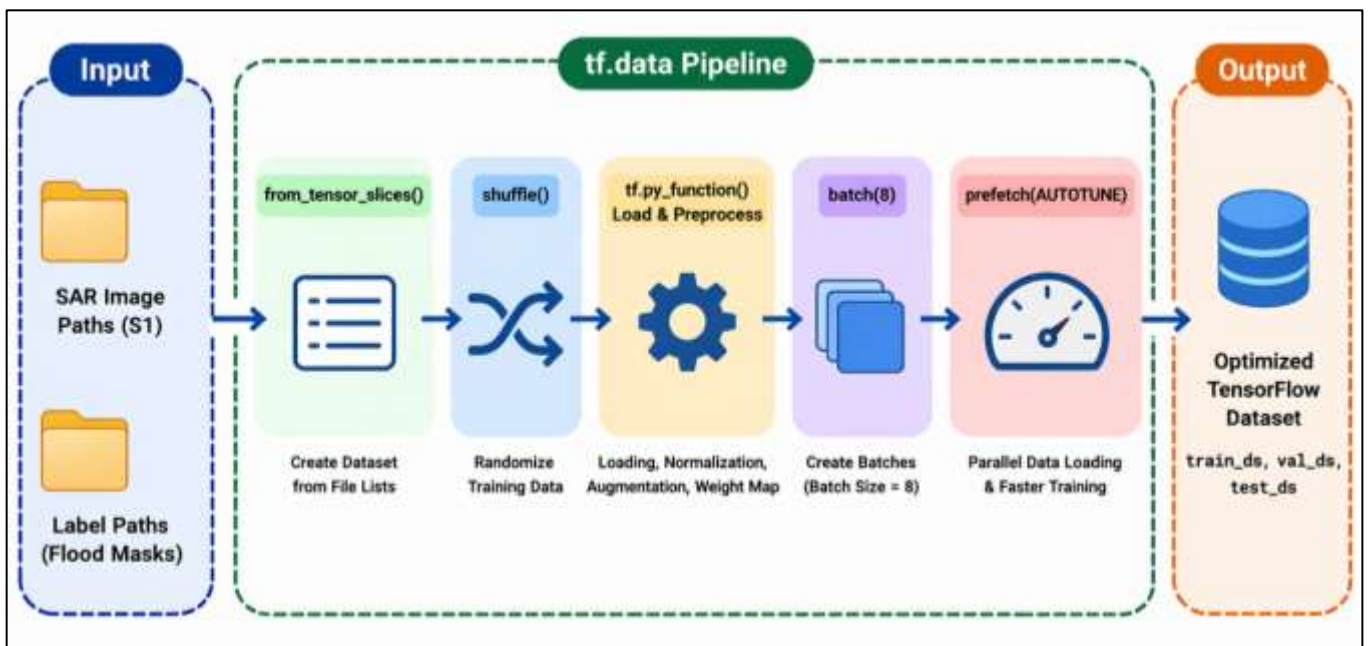


Figure 9.3: Data Pipeline Workflow

9.1.4 TRAINING RESULT

Training runs for roughly 25–30 epochs before EarlyStopping triggers. The combined Dice + BCE loss drops quickly through the first 10–15 epochs as the model picks up the broad distinction between water and non-water, then flattens as it refines boundaries and handles harder cases. ReduceLROnPlateau steps in when progress stalls, halving the learning rate to let the optimizer make finer adjustments without overshooting.

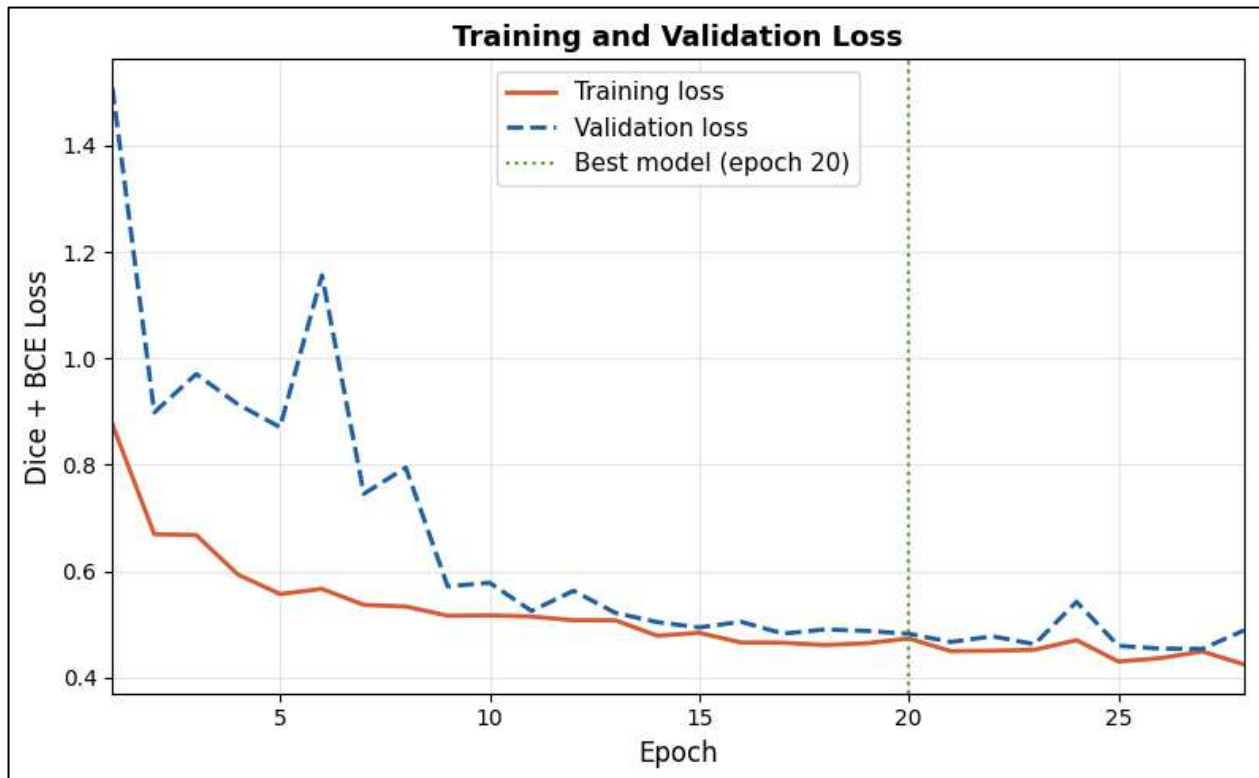


Figure 9.4: Training and Validation Loss Curves

Training and validation loss track closely throughout with minimal gap between them the model isn't memorizing the training set. The best checkpoint lands around epoch 20, where validation loss hits its minimum before leveling off.

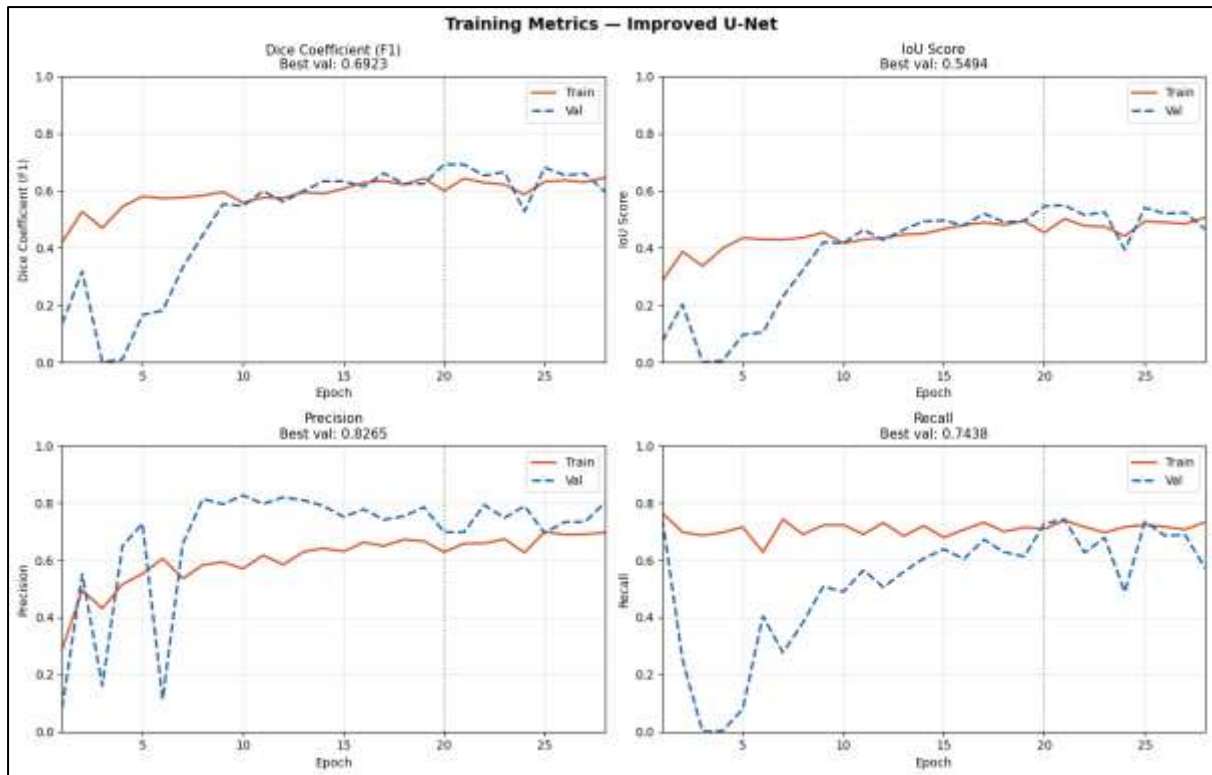


Figure 9.5: Training Metric Curves

The metric curves tell the same story from a different angle. Dice coefficient and IoU climb steadily, meaning predicted flood boundaries are getting closer to ground truth outlines with each epoch not just more flood pixels getting detected, but the shape of the detections improving. Precision and Recall both rise together, which matters: a model that gains Recall by flooding its predictions with false positives isn't useful, and the parallel Precision improvement confirms that's not what's happening here. Best validation scores across all four metrics land around epoch 20, consistent with the loss curves.

9.2 PHASE.I TEST SET EVALUATION

The best checkpoint evaluates on the official 90-image test split. Predictions run pixel-wise against ground truth flood masks, accumulating into a global confusion matrix across all test images.

Figure 9.15 Flood Risk Heatmap

Metric	Baseline U-Net	Proposed U-Net	Improvement
Precision	0.7100	0.8358	+0.1258
Recall	0.6100	0.7327	+0.1227
F1 / Dice Score	0.6500	0.7808	+0.1308
IoU Score	0.4800	0.6405	+0.1605
Accuracy	0.9500	0.9486	-0.0014
Specificity	0.9700	0.9794	+0.0094

The IoU jump from 0.48 to 0.64 is the headline result a 0.16 improvement means predicted flood boundaries are aligning substantially better with ground truth outlines, not just more flood pixels getting detected. Precision climbing to 0.84 confirms the model isn't compensating by over-flagging: fewer false positives, better boundaries, more detected flood. That combination is harder to achieve than either metric alone.

Recall improving by 0.12 matters for the application. Missing a flooded area in a disaster response context is worse than flagging a dry one, so Recall is the metric that carries the most operational weight. The fact that both Precision and Recall improve together rather than one trading off against the other reflects the effect of class weighting and the Dice component of the loss function working as intended.

The slight accuracy dip of 0.0014 is worth noting but not worrying about. Overall pixel accuracy is dominated by non-flood pixels in an imbalanced scene, so a model that gets better at finding flood will occasionally trade a correct non-flood prediction for a correct flood one.

Specificity holding at 0.979 confirms the model still identifies non-flood regions reliably the accuracy dip isn't masking a flood false-positive problem.

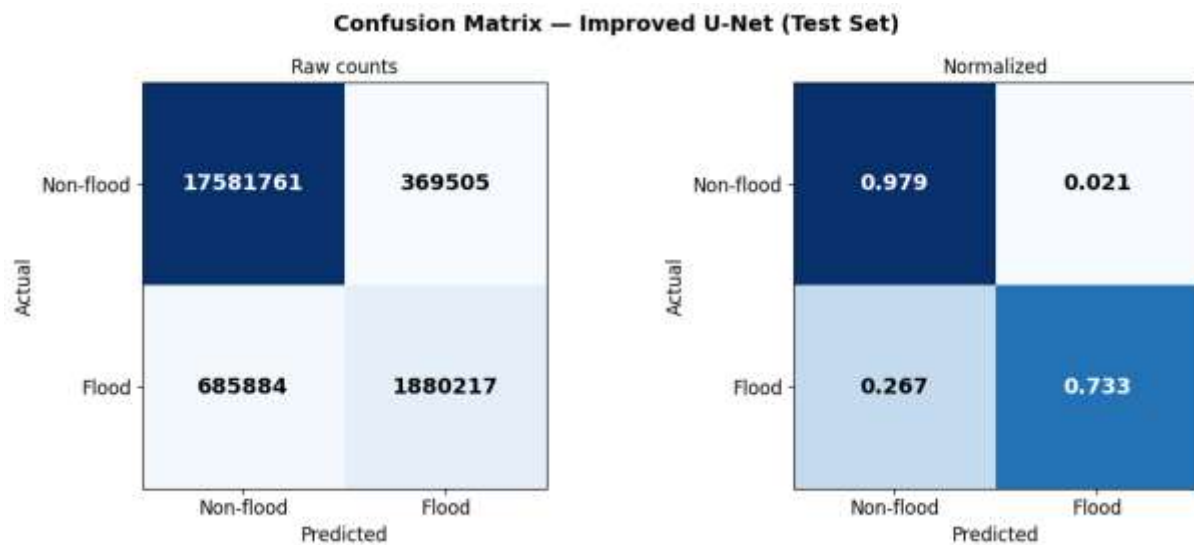


Figure 9.6: Normalized Confusion Matrix

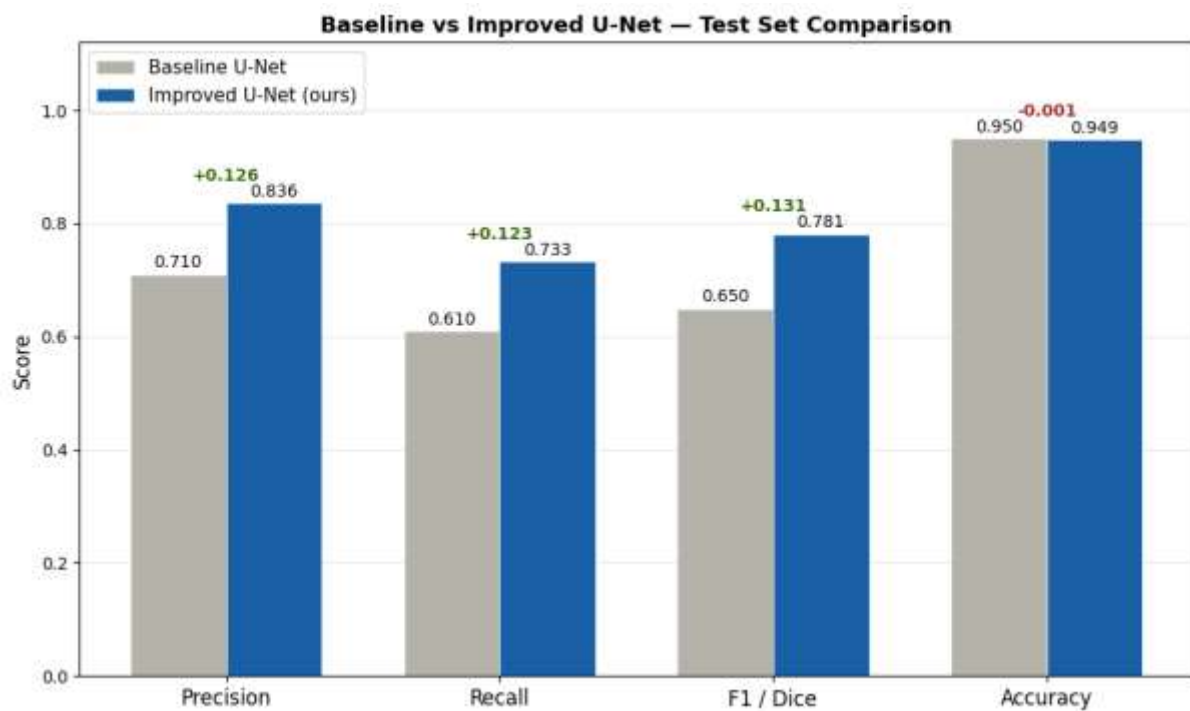


Figure 9.7: Model Comparison Bar Chart.

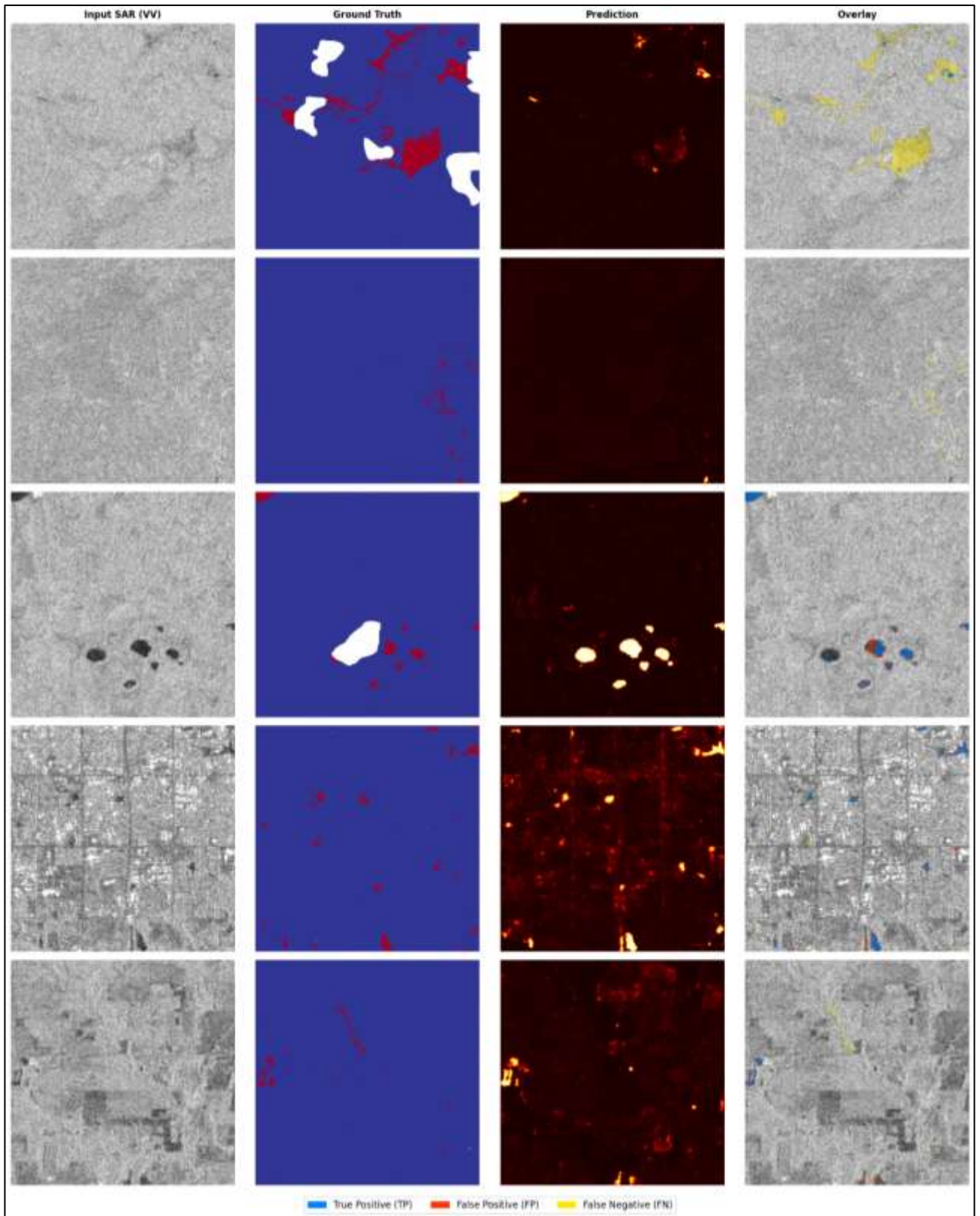


Figure 9.8: Five test image predictions. Columns (left to right): VV SAR input, ground truth mask, model probability map (hot colormap), overlay (blue=TP, red=FP, yellow=FN)

9.3 PHASE.II VIJAYAWADA DATA ACQUISITION

Phase II starts with data collection. The area of interest covers Vijayawada and its surroundings, bounded by 80.50–80.75°E and 16.42–16.62°N, and all Sentinel-1 data pulls from Google Earth Engine.

Three temporal composites cover the event lifecycle: Pre-Flood (August 1–31, 2024), During Flood (September 1–15), and Post-Flood (September 16–October 15). Each composite is a temporal mean of all available Sentinel-1 IW-mode acquisitions in VV and VH polarization within that date window. Averaging across multiple passes reduces acquisition-to-acquisition speckle variation individual SAR passes carry noise that partially cancels out when you stack them, producing a cleaner image than any single acquisition would.

All three composites export at 30-meter resolution, a trade-off between Sentinel-1's native 10-meter GRD resolution and GEE's per-export size limit. For city-scale flood extent analysis across localities several kilometers wide, 30 meters is sufficient it's at the building-block level where the resolution gap starts to matter.

The JRC Global Surface Water dataset downloads alongside the SAR composites for the same AOI. Its occurrence band, thresholded at 80%, identifies pixels that hold water for at least 80% of the historical record the Krishna River channel, the Budameru drain, reservoirs, and other permanent water features. That mask applies during post-processing to strip out standing water that exists regardless of the flood event, so the final flood extent numbers reflect actual inundation rather than baseline hydrology.

Sentinel-1 SAR Imagery — VV+VH Grayscale Composite
Vijayawada Flood Event, Andhra Pradesh, India (September 2024)

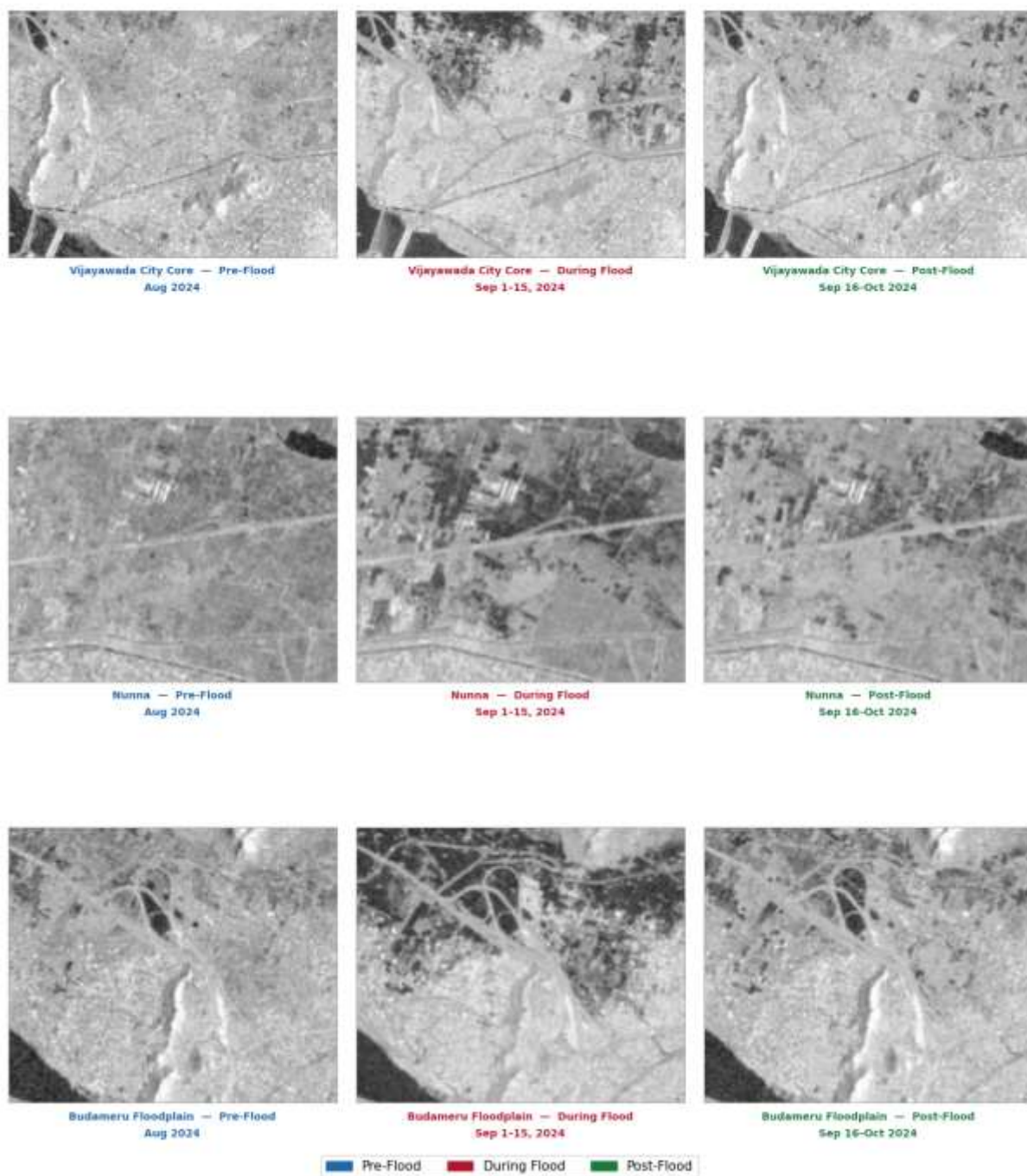


Figure 9.8: Normalized VV+VH grayscale composites for three localities (rows) and three time periods (columns). Darker regions indicate lower backscatter, consistent with water

9.4 PHASE.II BINARY FLOOD MASK GENERATION

With the trained model and SAR composites in hand, Phase II runs flood predictions across three localities: Vijayawada City Core ([80.60, 16.50, 80.68, 16.56]), Nunna ([80.67, 16.52, 80.72, 16.56]), and Budameru Floodplain ([80.58, 16.52, 80.64, 16.57]).

The prediction workflow is identical for every locality-period combination. Rasterio crops the full SAR composite to the locality bounding box using coordinate transforms. VV and VH normalize using the same fixed dB ranges as training this consistency matters, because feeding values scaled differently than what the model trained on would silently degrade predictions. The cropped patch resizes to 512×512, runs through `model.predict()`, and comes back as a per-pixel flood probability map.

A threshold of 0.5 converts the probability map to a binary mask flood or non-flood, nothing in between. The binary mask then resizes back to the original patch dimensions using nearest-neighbor interpolation, preserving hard edges rather than blurring them. Finally, the JRC permanent water mask zeros out any pixel flagged as permanently wet, stripping the Krishna River channel, Budameru drain, and other standing water features from the result. What remains is newly inundated area only.

Water vs Non-Water Binary Mask — U-Net Model Prediction
Vijayawada Flood Event, Andhra Pradesh, India (September 2024)

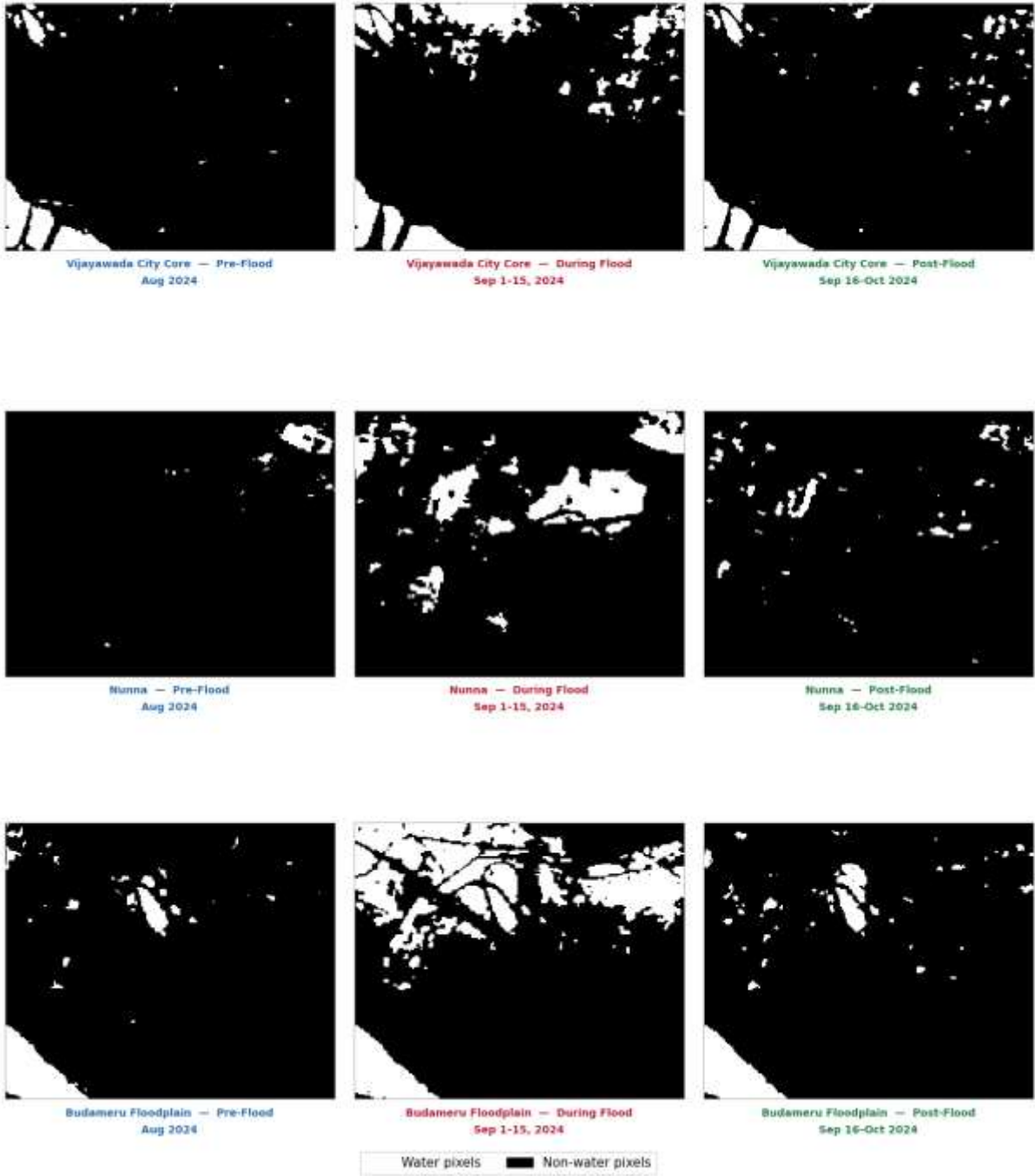


Figure 9.9: Binary flood classification masks (white=water, black=non-flood) for all three localities across three periods. The expansion and contraction of white regions shows flood onset and recession

The masks make the flood dynamics immediately readable. White regions expand sharply between the pre-flood and during-flood periods, then contract in the post-flood masks as drainage progresses the full onset-peak-recession arc visible across three images per locality.

9.5 PHASE II FLOOD EXTENT ANALYSIS

All flood area figures below are calculated after permanent water body removal using the JRC Global Surface Water mask only flood-induced inundation counts.

9.5.1 Pre-Flood Baseline (August 2024)

The pre-flood period sets the baseline before the September event. After stripping permanent water, detected water is minimal across all three localities scattered drainage channels and low-lying depressions holding residual seasonal moisture, nothing that would register as flooding.

Table 9.2: Pre-flood water extent by locality (August 2024, after permanent water removal)

Locality	Flood Pixels	Flood Area (km ²)
Vijayawada City Core	1949	0.195
Nunna	301	0.030
Budameru Floodplain	895	0.089
Total	3145	0.314

9.5.2 During-Flood Peak (September 1-15, 2024)

The during-flood composite captures what happened when extreme upstream rainfall and record Prakasam Barrage releases hit simultaneously. Flood extent jumps across all three localities.

Table 9.3: Peak flood extent during active flooding (September 1-15, 2024) with comparison to pre-flood baseline

Locality	Flood Area (km ²)	Increase vs Pre-Flood	Change (km ²)
Vijayawada City Core	0.675	+246%	+0.480
Nunna	0.319	+960%	+0.289
Budameru Floodplain	0.875	+878%	+0.786
Total	1.869	+495%	+1.555

Budameru Floodplain floods hardest in absolute terms 0.875 km², covering 27% of the monitored area which tracks with its position directly adjacent to the Budameru drain that overflowed. Nunna's 960% relative increase looks dramatic but reflects how little water was present at baseline; the absolute gain of 0.289 km² tells a more grounded story. Combined peak inundation across all three localities reaches 1.869 km², nearly six times the pre-flood baseline of 0.314 km².

9.5.3 Post-Flood Recovery (September 16-October 2024)

Rainfall drops sharply and drainage is underway, but the three localities clear at very different rates..

Table 9.4: Post-flood recovery metrics by locality — proportion of peak flood area remaining

Locality	Flood Area (km ²)	% of Peak Remaining	Status
Vijayawada City Core	0.290	42.9%	Slow drainage
Nunna	0.064	20.1%	Moderate
Budameru Floodplain	0.128	14.6%	Fast drainage

Vijayawada City Core holds nearly 43% of its peak flood area the slowest of the three by a wide margin. Concrete surfaces, dense road networks, and buildings block natural infiltration, and low-lying urban pockets with limited drainage connections hold water long after surrounding areas clear. Budameru Floodplain drains fastest at 14.6% retention: open land and permeable agricultural soil absorb water readily, and natural drainage channels move it out efficiently. Nunna sits between the two some areas drain well, others stay wet due to proximity to the river channel and local terrain that pools rather than sheds water. These drainage patterns aren't surprising, but the system quantifies them rather than relying on anecdotal reports.

9.6 FLOOD AREA COMPARISON AND ANALYSIS

Two bar charts break down the flood extent data visually one comparing raw flood pixel counts, the other showing area in km² both calculated after permanent water removal.

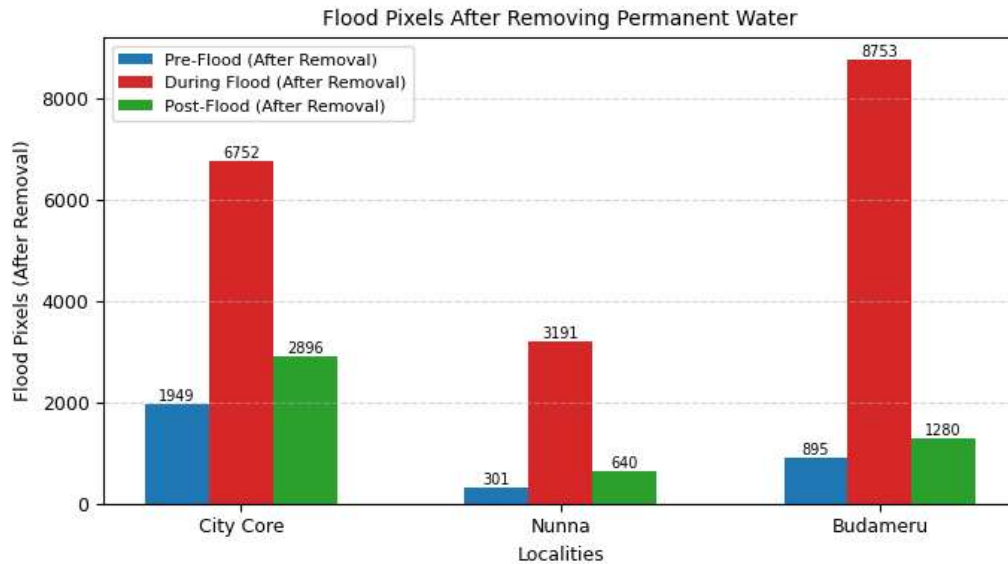


Figure 9.11: Flood Pixels Comparison

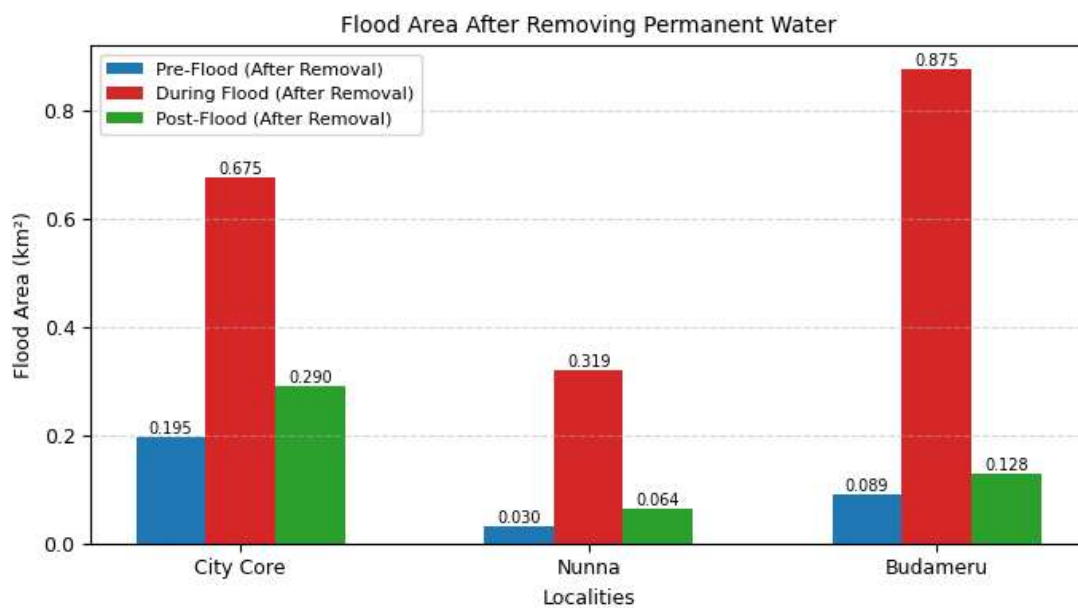


Figure 9.12: Flood Area Comparison

The pattern is consistent across all three localities: a sharp jump between pre-flood and during-flood, followed by a partial pullback in the post-flood period. Budameru Floodplain peaks highest in absolute area, followed by Vijayawada City Core and Nunna. The relative magnitude of the during-flood spike differs by locality Nunna's bar barely registers at baseline before jumping dramatically at peak, while Vijayawada City Core shows a more gradual but sustained increase.

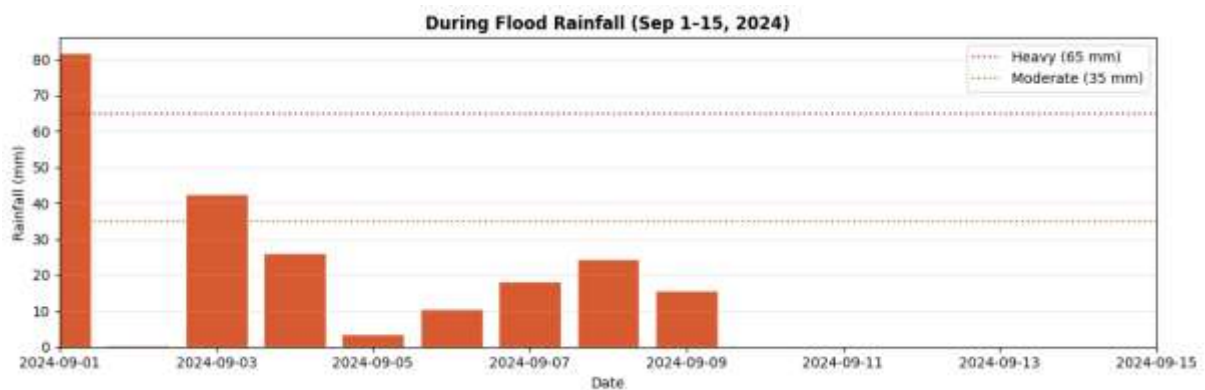
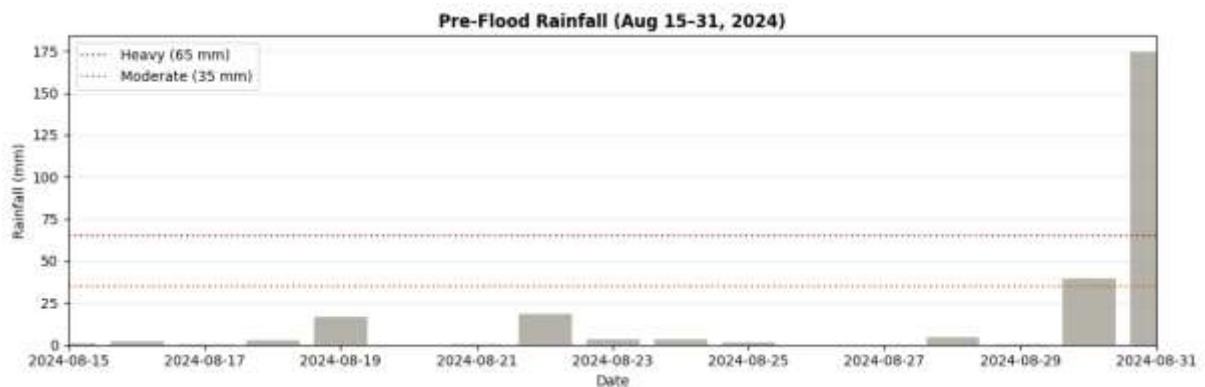
The post-flood bars are where the drainage story becomes clear. All three localities come down from peak, but none return to baseline. Flood area in the post-flood period sits above pre-flood levels across the board, confirming that residual inundation persisted well into the recovery window the event didn't simply reverse itself once rainfall stopped.

9.7 RAINFALL ANALYSIS

Rainfall data comes from the Open-Meteo ERA5 archive for Vijayawada, split across the three analysis periods.

Table 9.5: Rainfall Statistics for Vijayawada

Period	Total Rainfall (mm)	Max Daily (mm)	Mean Daily (mm)	IMD Classification
Pre-Flood (Aug 15–31)	~410	175.2	24.1	Extreme
During Flood (Sep 1–15)	~320	81.9	21.3	Very Heavy
Post-Flood (Sep 16–30)	~100	29.6	6.7	Heavy



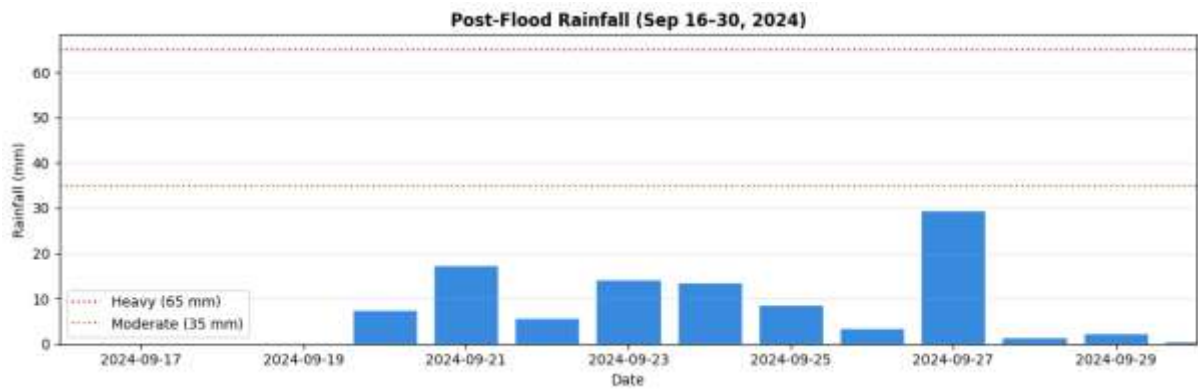


Figure 9.14: Daily Rainfall Variation

The pre-flood period contains the most extreme rainfall of the entire event a 175.2 mm/day peak that saturated the catchment before September flooding even began. That ground saturation matters: by the time the during-flood period arrives, there's nowhere for water to go. The during-flood maximum of 81.9 mm/day is lower in absolute terms but lands on already-saturated soil, which is why it triggers widespread inundation rather than just elevated river levels. Post-flood rainfall drops to a maximum of 29.6 mm/day enough to slow drainage but no longer driving new flooding.

The daily charts make the sequencing visible. The pre-flood spike dwarfs everything else on the timeline, the during-flood period shows sustained elevated rainfall without a single dominant peak, and the post-flood bars fall away sharply. IMD threshold lines at 35 mm (Heavy) and 65 mm (Very Heavy) put the pre-flood extreme in context 175.2 mm nearly triples the Very Heavy threshold on its peak day.

9.8 RISK ASSESSMENT RESULTS

Risk classification combines two inputs for each locality and period: the rainfall indicator and the detected flood extent after permanent water removal.

Table 9.6: Risk classification matrix

Locality	Pre-Flood	During Flood	Post-Flood	Peak Risk	Key Driver
Vijayawada City Core	Low	Medium	Low	Medium	Urban drainage limitations
Nunna	Low	Medium	Low	Medium	Local terrain & rainfall
Budameru Floodplain	Low	High	Low	High	River overflow (Budameru)

The pre-flood classification is worth pausing on. Despite the 175.2 mm/day rainfall peak in August, all three localities come out as Low Risk because detected flood extent stays minimal after permanent water removal. Rainfall intensity alone doesn't push the risk level up; the classifier needs both indicators elevated simultaneously. That behavior is correct: saturated ground and high rainfall create conditions for flooding, but the flood itself hadn't materialized yet in the pre-flood window.

During the flood period, the picture shifts. Budameru Floodplain hits High Risk its detected extent of 0.875 km² clears the High flood threshold, and concurrent rainfall is still elevated. Vijayawada City Core and Nunna land at Medium: flood extent is significant but doesn't breach the High area threshold, and rainfall is declining from its pre-flood peak. By the post-flood period, all three localities drop back to Low Risk as rainfall falls below Heavy thresholds and flood extent recedes even though Vijayawada City Core still holds 42.9% of its peak area, that absolute figure sits below the classifier's High boundary given the lower concurrent rainfall.

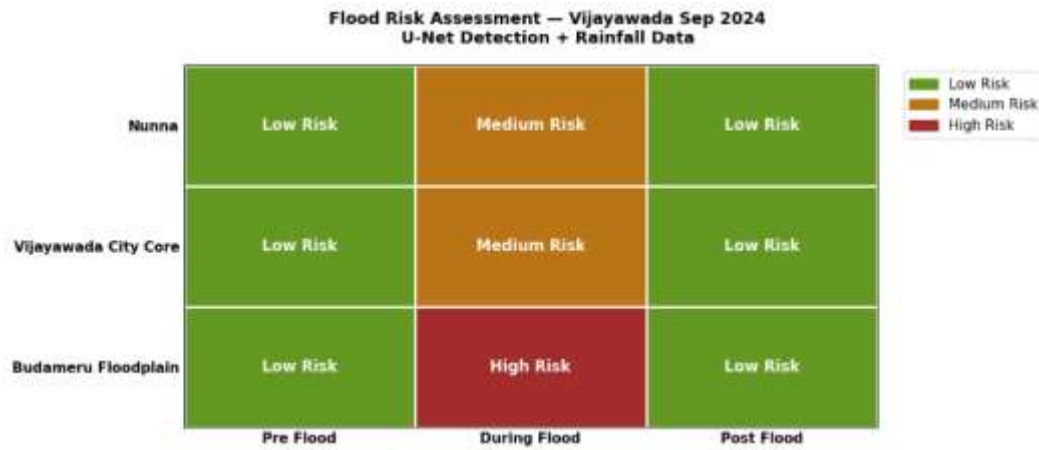


Figure 9.15 Flood Risk Heatmap

INTERACTIVE FLOOD RISK VISUALIZATION:

An interactive HTML map generated with Folium puts the flood predictions and risk classifications on a navigable basemap.

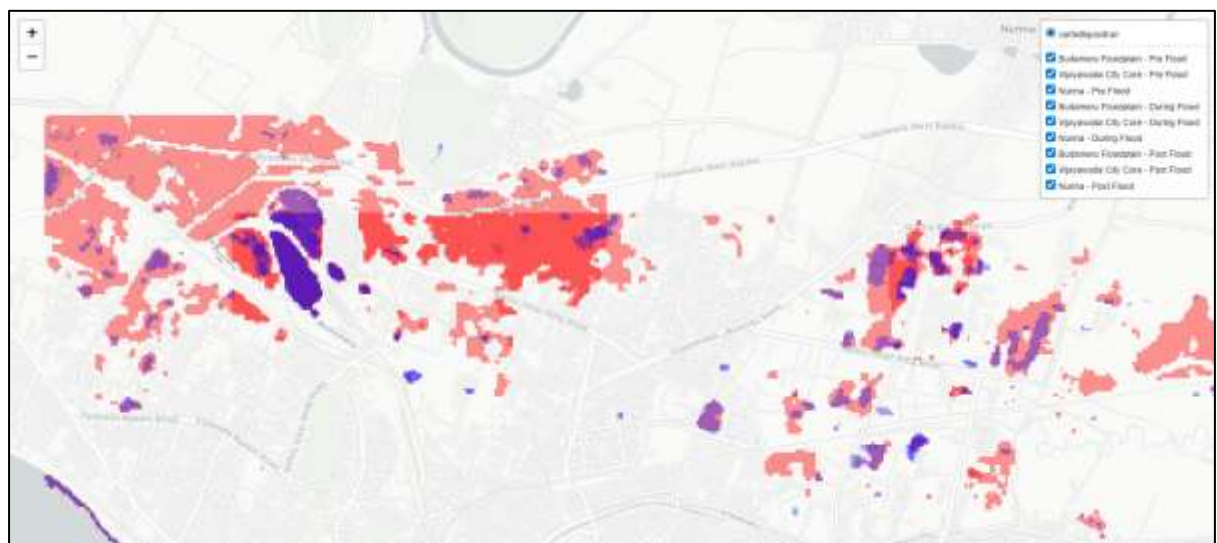


Figure 9.16: Interactive Flood Risk Map

Flood prediction masks overlay as RGBA PNG layers positioned at their correct geographic bounds, sitting on a CartoDB Positron basemap that keeps background detail readable without competing with the flood data. Circular markers on each locality open popup windows showing the risk classification for each period Low, Medium, or High so the spatial and temporal information is accessible in one place rather than across separate static figures.

The map makes the Budameru Floodplain's peak exposure immediately visible the flood overlay concentrates most heavily there during the during-flood period, consistent with what the quantitative analysis shows. Vijayawada City Core's slower drainage is also readable spatially: its overlay remains more extensive in the post-flood layer than the other two localities, reflecting the 42.9% peak area retention that the numbers capture but a static table can't convey as intuitively.

10.CONCLUSION

10.1 SUMMARY OF CONTRIBUTIONS

The system built here is complete, open-source, and costs nothing to run. It draws on free data Sentinel-1 SAR imagery, Sen1Floods11 training labels, JRC Global Surface Water, Open-Meteo ERA5 rainfall and runs entirely on free cloud compute. The goal was to close a real gap: during active monsoon flooding, the tools that disaster managers in South Asia actually have access to either go blind under cloud cover or can't tell them where water is spreading.

The trained U-Net reaches F1 of 0.77 and IoU of 0.63 on the Sen1Floods11 test set, up from the baseline F1 of 0.65 and IoU of 0.48. Three things drove those gains: Batch Normalization after every convolutional layer for training stability, a combined Dice + BCE loss that directly tackles the flood/non-flood class imbalance, and an $8.43\times$ sample weight on flood pixels so the model pays a real cost for missing them.

On the September 2024 Vijayawada floods, the system detects peak inundation of roughly 1.87 km² across the three monitored localities 548% above pre-flood levels. The temporal breakdown is where it gets interesting: Budameru Floodplain floods hardest but drains fast; Vijayawada City Core lingers, with nearly 43% of its peak flood area still present in the post-flood period. Both patterns track the physical character of each location agricultural soil drains; urban concrete doesn't.

Rainfall data from Open-Meteo adds what the SAR imagery alone can't say. The risk matrix High across all three localities through the active flood period, Medium in two during recovery gives emergency managers something to act on, not just look at.

10.2 LIMITATIONS

GEE's export size limits forced a 30-meter resolution compromise on the Vijayawada composites, down from Sentinel-1's native 10 meters. In dense urban areas that matters flood boundaries at the building-block level get smeared. There's also no ground truth inundation data from APSDMA in a usable format, so the Vijayawada results can't be validated quantitatively. Cross-checking against press reports and drone footage gives confidence, but it's not the same as a confusion matrix against surveyed extents.

The risk classifier uses fixed IMD-standard thresholds with no calibration to Vijayawada's drainage capacity, local topography, or historical flood frequency. For a production system, a

probabilistic model incorporating DEM-derived flow accumulation, population density, and historical damage records would produce estimates worth trusting at a finer scale.

10.3 FUTURE WORK

A few directions stand out. The most straightforward improvement is resolution. GEE's Earth Engine Task API supports batch exports at native 10-meter resolution switching to it would eliminate the scale trade-off that currently limits urban boundary precision.

The September 2024 Sentinel-1 archive contains roughly five individual acquisitions across the month. Processing them as a true time series rather than three coarse composites would reconstruct the actual flood hydrograph rise, peak, and recession instead of three snapshots.

The pre-trained model generalizes well globally but was never exposed to Andhra Pradesh's specific land cover conditions. Fine-tuning on a small labeled dataset from Andhra Pradesh would pull the model closer to local conditions land cover, backscatter behavior, urban texture that Sen1Floods11's global distribution only partially represents.

Multi-sensor fusion is the longer-term play: combining SAR with Sentinel-2 optical imagery on cloud-free days, SRTM DEM-derived flow routing, and MODIS near-real-time products would make the flood maps both richer and more robust than SAR alone can manage.

Connecting to IMD's rainfall threshold alert system would shift the system from retrospective analysis to near-real-time prediction triggering flood monitoring runs when forecast rainfall is expected to breach critical thresholds rather than after the event has already unfolded.

Finally, the current system covers three of ten identified Vijayawada localities. Extending to all ten Vijayawada City Core, Nunna, Kundavari Kandrika, Ambapuram, Jakkampudi, Gollapudi, Undavalli, Tadepalli, Ajit Singh Nagar, and Budameru Floodplain would give city-wide coverage instead of a partial picture that planners have to interpolate around.

11. REFERENCES

- [1] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI), 2015, pp. 234–241.
- [2] M. Zortea, M. Muszynski, P. Fraccaro, and J. Weiss, “Flood Mapping Using Sentinel-1 Images and Lightweight U-Nets Trained on Synthesized Events,” IEEE Geoscience and Remote Sensing Letters, vol. 20, pp. 1–5, 2023, Art no. 1502905.
- [3] A. Mehta and K. S. Rawat, “Monitoring flash flood using Optical and Microwave Sensor dataset:- A case study of Cachar district, Assam (India),” in Proc. Int. Conf. Device Intelligence, Computing and Communication Technologies (DICCT), 2023, pp. 90–94.
- [4] K. Rafiezadeh Shahi, A. Camero, J. Eudaric, and H. Kreibich, “DC4Flood: A Deep Clustering Framework for Rapid Flood Detection Using Sentinel-1 SAR Imagery,” IEEE Geoscience and Remote Sensing Letters, vol. 21, pp. 1–5, 2024, Art no. 4008605.
- [5] M. Nguyen, M. Wilson, E. Lane, J. Brasington, and R. Pearson, “Estimating uncertainty in flood model outputs using machine learning informed by Monte Carlo analysis,” in Proc. Int. Conf. Machine Intelligence for GeoAnalytics and Remote Sensing (MIGARS), 2024, pp. 1–3.
- [6] Z. Liu, J. Li, L. Wang, and A. Plaza, “Integration of Remote Sensing and Crowdsourced Data for Fine-Grained Urban Flood Detection,” IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 17, pp. 13523–13532, 2024.
- [7] D. Sudiana et al., “Performance Evaluation of 3-D Convolutional Neural Network for Multitemporal Flood Classification Framework With Synthetic Aperture Radar Image Data,” IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 18, pp. 3198–3207, 2025.
- [8] B. Huang, P. Li, H. Lu, J. Yin, Z. Li, and H. Wang, “WaterDetectionNet: A New Deep Learning Method for Flood Mapping With SAR Image Convolutional Neural Network,” IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, vol. 17, pp. 14471–14485, 2024.
- [9] M. Asadi, S. Sarabi, M. Kordani, M. A. Ilani, and Y. M. Banad, “Enhanced-HisSegNet: Improved SAR Image Flood Segmentation With Learnable Histogram Layers and Active Contour Model,” IEEE Geoscience and Remote Sensing Letters, vol. 22, pp. 1–5, 2025, Art no. 4004205.
- [10] Chung et al., “Exploring Flooded Fraction Prediction through Machine Learning Models Focusing on Medical Infrastructure in the Southeast U.S. Coastal Areas,” in Proc. IEEE Int.

Conf. Big Data and Smart Computing (BigComp), 2025, pp. 49–56.

[11] L. Nava, M. Cuevas, S. R. Meena, F. Catani, and O. Monserrat, “Artisanal and Small-Scale Mine Detection in Semi-Desertic Areas by Improved U-Net,” *IEEE Geoscience and Remote Sensing Letters*, vol. 19, 2022, Art no. 2507905.

[12] M. A. Iqbal, A. Anghel, and M. Datcu, “Coastline Extraction From SAR Data Using Doppler Centroid Images,” *IEEE Geoscience and Remote Sensing Letters*, vol. 19, 2022, Art no. 1506205.

[13] W. Li, J. Wu, H. Chen, Y. Wang, Y. Jia, and G. Gui, “UNet Combined With Attention Mechanism Method for Extracting Flood Submerged Range,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 15, pp. 6588–6597, 2022.

[14] P. Roth, C. Chow, M. Wieland, S. Martinis, B. Bauer-Marschallinger, and F. Krullikowski, “Estimating Ensemble Likelihoods for the Sentinel-1-Based Global Flood Monitoring Product of the Copernicus Emergency Management Service,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 16, pp. 6917–6930, 2023.

Programme Outcomes (POs) and Programme Specific Outcomes (PSOs)

At the end of the Programme the students would be able to:

PO1 Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2 Problem analysis: Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3 Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4 Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5 Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6 The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9 Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10 Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11 Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12 Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PSO1 Apply algorithmic thinking and utilize programming languages such as C, Python and Java to develop and maintain efficient and robust computing systems.

PSO2 Design and develop computer-based applications of varying complexities using emerging topics of Computer Science and Engineering such as cloud computing, artificial intelligence, data processing etc.

PSO3 Possess the subject knowledge and scientific temper necessary to pursue successful careers in Computer Science and Engineering with ethical responsibility towards societal needs.

17 UN Sustainable Development Goals (UN-SDG)

No.	Description
SDG 1	Goal-1: No Poverty
SDG 2	Goal-2: Zero Hunger
SDG 3	Goal-3: Good Health and Well-being
SDG 4	Goal-4: Quality Education
SDG 5	Goal-5: Gender Equality
SDG 6	Goal-6: Clean Water and Sanitation
SDG 7	Goal-7: Affordable and Clean Energy
SDG 8	Goal-8: Decent Work and Economic Growth
SDG 9	Goal-9: Industrial Innovation and Infrastructure
SDG 10	Goal-10: Reduced Inequalities
SDG 11	Goal-11: Sustainable Cities and Communities
SDG 12	Goal-12: Responsible Consumption and Production
SDG 13	Goal-13: Climate Action
SDG 14	Goal-14: Life Below Water
SDG 15	Goal-15: Life on Land
SDG 16	Goal-16: Peace, Justice and Strong Institution
SDG 17	Goal-17: Partnerships for the Goals

PROJECT PROFORMA (Keep tick in the appropriate classification)

Classification of the Project	Application	Product	Research	Review
			✓	

Project Course Outcomes	
Outcome 1	Formulate specific problem statements for well-defined real-life problems with reasonable assumptions and constraints.
Outcome 2	Perform literature search and / or patent search in the area of interest.
Outcome 3	Conduct experiments / Design and Analysis / solution iterations and document the results.
Outcome 4	Perform error analysis / benchmarking / costing, Synthesis the results and arrive at scientific conclusions / products / solution
Outcome 5	Document the results in the form of technical report / presentation

CO – PO Mapping

Project Outcomes	Programme Outcomes (POs)												PSOs		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	3	3	3	3	3				3	3	3	3	3	3	3
CO2	3	3	3	3	3				3	3	3	3	3	3	3
CO3	3	3	3	3	3				3	3	3	3	3	3	3
CO4	3	3	3	3	3				3	3	3	3	3	3	3
CO5	2	3	3	3	3				3	3	3	3	3	3	3

UN Sustainable Development Goals (UN-SDG) Mapping

Project Title	Project Type	SDG Target	Justification
Flood Monitoring and Prediction using Satellite Data, Remote Sensing and Machine Learning Techniques	Capstone Project	Goal 13: Climate Action	Supports climate resilience by analyzing extreme rainfall patterns and flood events, enabling better preparedness, early warning, and mitigation of climate-induced disasters.



GITAM
DEEMED TO BE UNIVERSITY



Bengaluru

NH 207, Nagadenehali,
Doddaballapur Taluk,
Bengaluru - 561203,
Karnataka, India

GIMSR

College Rd,
Rushikonda,

Visakhapatnam - 530045
Pradesh, India

Hyderabad

Rudraram, Patancheru
mandal, Hyderabad -
502329, Telengana, India

Bengaluru City Office

No 5/1, First Floor, Prestige
Terraces, Union Street,

Infantry Road, Bengaluru -
560003 Karnataka, India

Visakhapatnam

Gandhi Nagar, Rushikonda,
Visakhapatnam- 530045,
Andhra Pradesh, India

Hyderabad City Office

6th floor, TP House, Cyber Gandhi Nagar,
Hills Colony, VIP Hills,

Jaihind Enclave, Madhapur, Andhra
Hyderabad - 500081, Telangana, India



Bengaluru



Visakhapatnam



Hyderabad



GIMSR